



FACULTE DE GENIE ELECTRIQUE ET D'INFORMATIQUE
DEPARTEMENT AUTOMATIQUE

MEMOIRE DE MAGISTER

en Automatique

Option : **Traitement d'Images et Reconnaissance de Formes**

présenté par

AMROUCHE Hakima

Ingénieur UMMTO

Sur l'hybridation des métaheuristiques.

Mémoire soutenu le: / /2012

devant le jury d'examen composé de :

BENFDILA Arezki

Professeur à l'U.M.M.T.O

Président

DIAF Moussa

Professeur à l'U.M.M.T.O

Rapporteur

HAMMOUCHE Kamal

M.C.A à l'U.M.M.T.O

Examineur

OUANES Mohand

M.C.A. à l'U.M.M.T.O.

Examineur

MAIDI Ahmed

M.C.A. à l'U.M.M.T.O.

Examineur

Résumé : Dans cette dernière décennie, les chercheurs visent à résoudre de nombreux problèmes difficiles rencontrés dans différents domaines en utilisant des métaheuristiques. Ces méthodes qui sont devenues très populaires grâce à leur simplicité de mise en œuvre sont en constante évolution d'où l'apparition de leur hybridation que ce soit entre elles ou avec d'autres méthodes d'optimisation. Cette technique est de plus en plus publiée sans toute fois justifier les raisons de l'hybridation. A travers ce mémoire, nous allons présenter un état de l'art sur les différentes techniques d'hybridation des métaheuristiques en se basant sur des travaux effectués en essayant de justifier les méthodes choisies pour effectuer cette l'hybridation. Pour bien illustrer nos résultats, une application d'une méthode hybridée de deux métaheuristiques (PSO-RS) est effectuée sur la segmentation d'images.

Mots clés : Optimisation, Métaheuristique, hybridation, les essaims de particules(PSO), le recuit simulé(RS).

Avant-propos

Le travail que nous présentons dans ce mémoire a été effectué au sein de laboratoire « Robotique et Vision » du département automatique, faculté de Génie Electrique et d'Informatique, université Mouloud Mammeri, Tizi-Ouzou.

J'exprime mes remerciements et ma gratitude à mon directeur de mémoire, Monsieur **DIAF Moussa**, professeur à l'Université Mouloud Mammeri, pour l'aide et le soutien et les conseils qu'il m'a apportés tout au long de ce travail

Nous ne manquerons pas de remercier Monsieur **BENFDILA Arezki**, Professeur à l'UMMTO pour avoir fait l'honneur de présider le jury de ce mémoire

Nous exprimons notre reconnaissance à Monsieur **HAMMOUCHE Kamal**, Maître de Conférences classe A, à l'UMMTO pour avoir accepté de faire partie du jury d'examen de notre mémoire.

Nous tenons à remercier Monsieur **OUANES Mohand**, maître de conférences classe A à l'Université Mouloud Mammeri pour avoir bien voulu faire partie du jury de ce mémoire.

Nous tenons également à remercier Monsieur **MAIDI Ahmed**, maître de conférences classe A à l'Université Mouloud Mammeri, pour avoir bien voulu faire partie du jury de ce mémoire.

Sommaire

Introduction générale

Chapitre 1 : Généralités sur les métaheuristiques

1. Introduction.....	1
2. Les métaheuristiques.....	3
3. Conclusion.....	16

Chapitre 2 : Hybridation de métaheuristiques

1. Introduction.....	18
2. Notion d'hybridation	19
3. Hybridation métaheuristiques/métaheuristiques	19
3.1. Classification hiérarchique des métaheuristiques.....	19
3.2. Classification à plat des métaheuristiques.....	23
4. Hybridation métaheuristiques/méthodes exactes.....	24
5. Discussion sur les classifications de méthodes hybrides.....	29
6. Etude de l'hybridation des essaims de particules	30
6.1. Hybridation de PSO avec les algorithmes évolutionnaires.....	30
6.2. Hybridation de PSO avec l'algorithme de l'évolution différentielle.....	32
6.3. Hybridation de PSO avec d'autres algorithmes	33
7. Conclusion.....	35

Chapitre 3 : Application d'une hybridation de deux métaheuristiques en segmentation d'images

1. Introduction.....	36
2. Segmentation d'images par classification des pixels.....	37
3. La méthode d'Otsu.....	39
4. Intégration la méthode d'Otsu dans l'algorithme RS	41
5. Intégration de la méthode d'Otsu dans l'algorithme PSO	42
6. Hybridation de l'algorithme PSO avec RS.....	43
7. Segmentation des images en fonction du nombre de classes.....	44
7.1 Segmentation des images en quatre classes.....	44
7.2 Segmentation des images en cinq classes.....	48

8. Conclusion.....	51
Conclusion générale	55
Référence bibliographique	

Introduction générale

Dans divers domaines économiques, le besoin croissant d'efficacité et de rentabilité a conduit les décideurs à solutionner leurs problèmes quelle que soit la difficulté en introduisant des méthodes d'optimisation. Cependant, la compétition entre les méthodes de recherche pour arriver aux meilleurs résultats en termes de qualité et de temps de traitement a conduit à distinguer un type de problèmes classés difficiles. Lorsque le problème à résoudre est NP-difficile, l'utilisation d'une méthode exacte qui garantit l'obtention de la meilleure solution nécessite un temps de calcul très long qui croît de façon exponentielle avec la taille du problème traité. Dans ce cas, l'utilisation de méthodes qui offrent une solution de bonne qualité, quasi optimale, mais en un temps raisonnable s'avère intéressante. Ces méthodes sont les métaheuristiques. Notant toutefois que, d'un point de vue algorithmique, les méthodes exactes reposent souvent sur l'utilisation d'une recherche arborescente en manipulant des configurations partielles et construisant la solution sans violer les contraintes auxquelles est soumis le problème traité. Du fait de la démarche constructive, ces méthodes ont plutôt tendance à intensifier la recherche dans une direction en étant guidée de proche en proche par les réductions de domaines dues aux contraintes. Par contre, les métaheuristiques manipulent généralement des configurations

complètes, sans garantir l'optimalité de la solution trouvée, en partant d'une configuration initiale où toutes les variables du problème sont instanciées. Ensuite, elles construisent cette solution de proche en proche jusqu'à la satisfaction d'un critère d'arrêt prédéfini. Ces méthodes fonctionnent selon une approche de recherche dite locale ou par voisinage. Lorsqu'on traite un ensemble de solutions en parallèle, l'approche employée est dite globale ou à population. Deux grands mécanismes ont été proposés pour améliorer l'efficacité de ce genre de méthode, à savoir, l'intensification et la diversification. L'intensification consiste à fouiller dans une zone précise de l'espace de recherche afin de trouver un optimum local alors que la diversification permet de se déplacer vers une autre zone de recherche non encore visitée. L'efficacité dépend généralement de l'alternance de ces deux mécanismes d'une façon intelligente. Pour améliorer les résultats concernant la recherche des optima, l'approche par une hybridation a été introduite. Dans la littérature, une activité intense existe depuis une dizaine d'années sur l'hybridation entre les éléments des métaheuristiques et entre les métaheuristiques et les méthodes exactes. Cependant, cette hybridation est effectuée sans aucune justification. A travers ce mémoire, une étude sur différentes techniques d'hybridation de métaheuristiques est effectuée en se basant sur des travaux déjà réalisés. Ainsi, dans le premier chapitre, un état de l'art sur les métaheuristiques les plus populaires est présenté.

Il est de même pour le second chapitre dans lequel est donné un état de l'art sur les techniques d'hybridation des métaheuristiques en se basant sur des travaux déjà effectués.

Dans le dernier chapitre, il s'agit pour nous de proposer une hybridation de deux métaheuristiques basées sur les essaims de

particules et le recuit simulé appliquées à la méthode de seuillage d'Otsu. Un ensemble de résultats analysés et commentés est présenté. Nous terminons ce mémoire par une conclusion générale qui récapitule les travaux de l'ensemble des chapitres et quelques perspectives majeures pour la poursuite de ce travail.

Chapitre 1

Généralités sur les métaheuristiques

1. Introduction

Dans la vie courante, nous sommes, souvent, confrontés à des problèmes qui peuvent être décrits sous forme d'un problème d'optimisation, comme le fait de minimiser les coûts de production pour accroître les bénéfices dans le milieu industriel. Dans ce type de problème, ceci a été rendu possible grâce, aussi, à la croissance de la puissance de calcul des ordinateurs produits ces vingt dernières années. L'optimisation combinatoire permet de minimiser ou maximiser des fonctions dans des systèmes dans lesquels peut intervenir un grand nombre de paramètres. Pour résoudre de tels problèmes, on définit un espace de recherche S fini et une fonction objectif f appelée aussi fitness ou fonction de coût qu'il s'agit d'optimiser, *ie.* minimiser ou maximiser. Cependant, pour les problèmes dits difficiles, on ne connaît pas d'algorithmes exacts rapides permettant de les résoudre. Même pour les autres, dits à variables continues, il n'existe pas, non plus, d'algorithmes permettant

de repérer un optimum global à coup sûr et en un nombre fini de calculs. En effet, en optimisation, le problème peut se présenter sous différentes appellations, selon sa complexité. Il peut être de classe P, s'il est facile ou résolu par l'application des méthodes classiques. Par contre, dans le cas d'existence de discontinuité, de fonction non dérivable et de présence de bruits, on parle de problèmes d'optimisation *difficile*. Ces derniers sont de deux types, à savoir, les problèmes combinatoires (ou problèmes discrets) comme le voyageur de commerce et les problèmes à variables continues. Dans le cas de méthodes linéaires à variables continues, la programmation linéaire est utilisée. Dans le cas où elles sont non linéaires, il s'agit d'une optimisation difficile continue. Notons, toutefois que, jusqu'à présent, ce sont, plutôt, les *méthodes combinatoires ou discrètes* regroupant les *méthodes exactes* comme la méthode du simplex [1], de Branch and Bound [2] et la programmation dynamique [3] qui sont utilisées. Néanmoins, lorsqu'on veut résoudre un problème complexe, ces méthodes prennent un temps de calcul qui croît exponentiellement avec la taille des instances du problème. Ceci conduit souvent à une explosion combinatoire. Pour éviter ce type de problème, on fait appel aux *méthodes approchées* qui ne cherchent pas forcément l'optimum absolu mais donnent une solution très proche de l'optimum absolu montrant l'inexistence d'une solution sensiblement meilleure et largement acceptée. Ceci est obtenu en utilisant les méthodes regroupant les heuristiques et les métaheuristiques. Nous rappelons que la plupart des heuristiques sont spécifiques à un problème donné et ne s'appliquent, en général, qu'à des problèmes discrets. Elles sont de complexité raisonnable, autrement dit, idéalement polynomiales mais efficaces et simples à mettre en œuvre. Quant aux métaheuristiques, ce sont des algorithmes stochastiques qui tentent de trouver l'optimum global d'un problème d'optimisation difficile en un

temps raisonnable sans être piégé dans des optima locaux. La figure 1.1 présente les différentes méthodes d'optimisation.

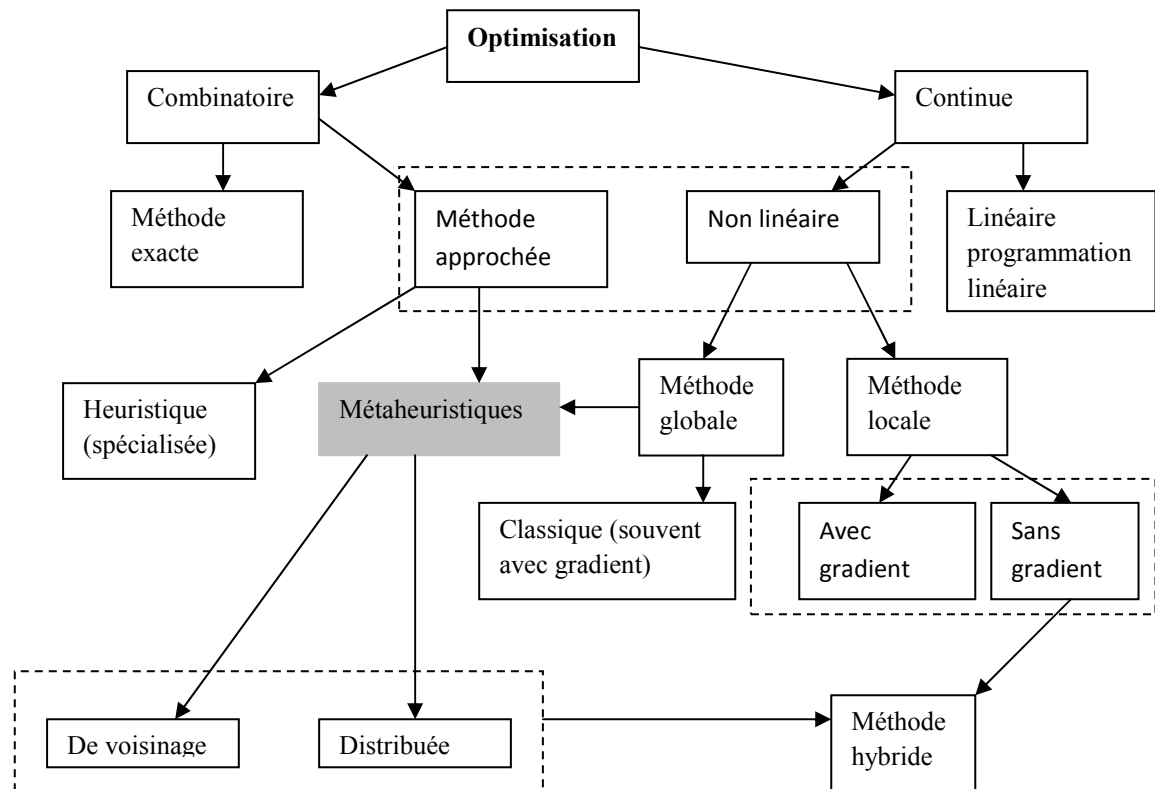


Fig.1.1 Présentation des différents algorithmes d'optimisation [4]

2. Les métaheuristiques

Apparues dans les années 1980, les métaheuristiques forment un ensemble d'algorithmes permettant de trouver la solution la plus rapide et la plus efficace pour une large gamme de problèmes d'optimisation difficile et pour lesquels on ne connaît pas de méthode classique plus efficace. Comme le montre la figure 1.2, il s'agit de trouver l'optimum global G sans être piégé par les autres optima locaux tel que le point L.

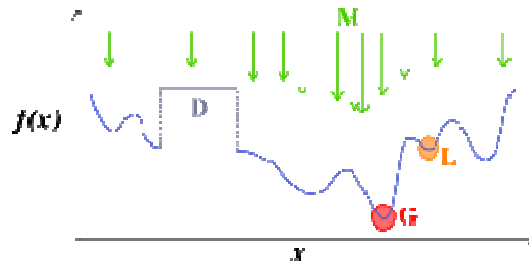


Fig.1.2 Représentation du minimum local et global d'une fonction

Les métaheuristiques fonctionnent selon un comportement itératif c'est-à-dire que le même schéma se reproduit un certain nombre de fois au cours de l'optimisation, généralement, elles s'articulent autour des trois notions suivantes :

- la diversification (exploration)
- l'intensification (exploitation)
- la mémorisation (apprentissage)

La diversification ou exploration désigne le processus qui dirige la procédure pour récolter de l'information sur le problème à optimiser. La stratégie de diversification la plus simple consiste à redémarrer périodiquement le processus de recherche à partir d'une solution générée aléatoirement ou choisie judicieusement dans une région non encore visitée de l'ensemble des solutions admissibles. Pour sa part, l'intensification ou exploitation utilise l'information déjà récoltée pour explorer, en détail, les zones jugées prometteuses dans l'espace de recherche. Sa mise en œuvre réside, le plus souvent, dans l'élargissement temporaire du voisinage de la solution courante. Quant à la mémorisation, elle est le support de l'apprentissage qui permet à l'algorithme de ne tenir compte que des zones où l'optimum global est susceptible de se trouver, évitant ainsi, les optima locaux qui sont de bonnes solutions, mais qui ne sont pas les meilleures des solutions possibles. Ainsi, en alternant l'intensification, la diversification et la mémorisation, le fonctionnement des métaheuristiques est progressif et itératif. L'étape initiale est souvent choisie de façon aléatoire et

l'étape d'arrêt est souvent fixée à l'aide d'un critère d'arrêt. Toutes les métaheuristiques s'appuient sur l'équilibre entre l'intensification et la diversification de la recherche. Sinon, on assistera à une convergence trop rapide vers des minima locaux par manque de diversification ou à une exploration trop longue par manque d'intensification.

Le fonctionnement des métaheuristiques est généralement inspiré à partir de systèmes physiques comme le recuit simulé, de systèmes biologiques comme les algorithmes évolutionnaires, de systèmes ethnologiques comme les algorithmes de colonies de fourmis ou de l'optimisation par essaims particulaires etc. Ainsi, les métaheuristiques peuvent être réparties en deux catégories à savoir, les méthodes à base de voisinage correspondant à la recherche locale et les méthodes à base de population correspondant à la recherche globale (fig.1.3).

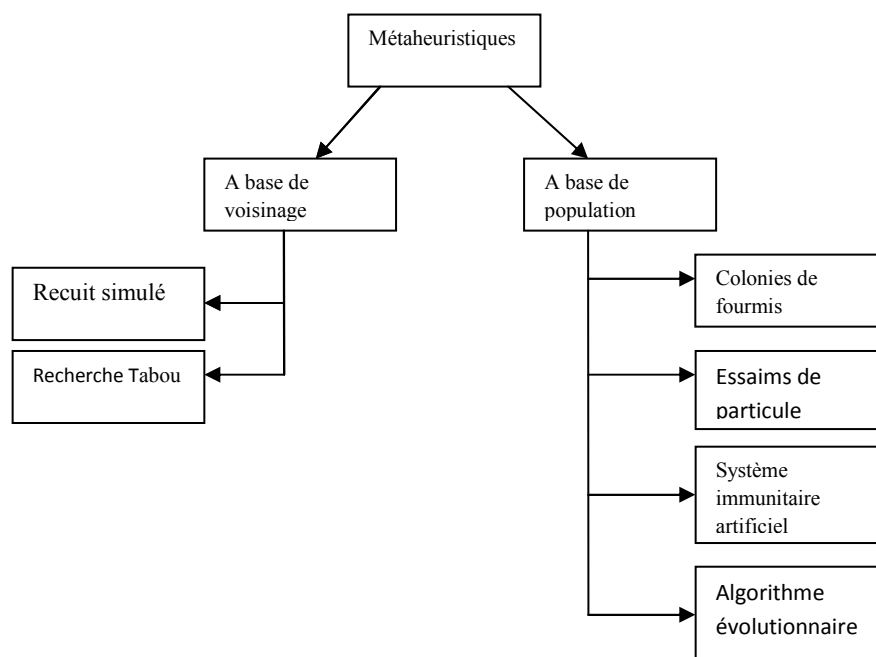


Fig.1.3 Les catégories des métaheuristiques.

Les métaheuristiques à base de voisinage ou à base de solution unique sont les méthodes de recherche locale. Elles sont itératives, *ie.*, à partir d'une solution unique x_0 considérée comme point de départ, la recherche consiste à passer d'une solution à une solution voisine par

déplacements successifs dans un voisinage constitué de l'ensemble des solutions. Souvent, les opérateurs de recherche locale s'arrêtent quand une solution localement optimale est trouvée. Mais accepter uniquement ce type de solution n'est pas toujours satisfaisant. Il est alors important de sortir de ces minima locaux en permettant à l'opérateur de recherche locale de trouver des points pour lesquels la nouvelle solution retenue sera de qualité meilleure que la précédente. C'est le principe adopté pour la méthode de descente, le recuit simulé, la recherche *tabou* et la méthode GRASP.

La **méthode de descente** appelée aussi méthode d'amélioration itérative ou Hill Climbing en anglais est assez ancienne [5]. Sa rapidité et sa simplicité font d'elle la méthode la plus utilisée. Elle part d'une solution initiale X_i d'un ensemble de recherche S et progresse vers une solution voisine X_{i+1} de meilleure qualité quel que soit i . Son algorithme général pour un problème de minimisation est donné à la figure 1.4.

Répéter : 1) Choisir X_{i+1} dans $V(X_i)$ 2) Si $f(X_{i+1}) < f(X_i)$ alors $X_i \leftarrow X_{i+1}$ 3) Jusqu'à ce que $f(X_{i+1}) \geq f(X_i)$, $\forall X_{i+1} \in S$ 4) Fin.
--

Fig.1.4 Algorithme de la méthode de descente.

Pour appliquer une descente, il faut bien choisir la solution initiale (X_i) qui est généralement aléatoire ou provenue d'une méthode approchée. Quand cette solution est aléatoire, on pourra appliquer plusieurs fois la descente en changeant à chaque fois la solution initiale et ne mémorisant que celle donnant le meilleur résultat. La méthode de descente est très facile à programmer. Son inconvénient réside dans le fait qu'elle calcule une solution locale car elle s'arrête au premier optimum rencontré. La répétition de déclaration de la solution initiale permet d'atténuer cet inconvénient.

La méthode **du recuit simulé** (Simulated Annealing) a été introduite par Kirkpatrick, Gelatt, et Vecchi en 1983 [6]. Cette métaheuristique s'inspire du recuit thermique des métaux en métallurgie. En effet, le refroidissement brutal d'un métal donne une structure amorphe, état métastable qui correspond à un optimum local pour un problème d'optimisation combinatoire. A l'opposé, si l'on le refroidit lentement, on obtient un état stable correspondant à un optimum global. Par analogie, on prend l'énergie du métal comme fonction objectif du problème combinatoire et les états physiques comme configurations aléatoires. La température est simulée par un paramètre de contrôle dépendant du problème traité. La méthode du recuit simulé autorise des variations de la configuration initiale ou solution initiale afin de s'échapper aux minima locaux dans lesquels le système peut être piégé lorsqu'on utilise la méthode de descente. Au lieu de refuser les solutions dégradant la fonction objectif, on les accepte avec une certaine probabilité en se basant sur le critère de Metropolis. Pour un problème de minimisation, à partir de la configuration courante i , on effectue une modification aléatoire j , puis on calcule l'énergie $\Delta E_{ij} = E_i - E_j$. Si cette énergie est augmentée, on accepte automatiquement cette modification sinon on tire aléatoirement un nombre entre 0 et 1 et on calcule la probabilité $\exp(-\Delta E_{ij}/T)$. Si cette probabilité est supérieure ou égale à ce nombre, on accepte la modification effectuée sinon elle est rejetée. Lorsque l'énergie reste stationnaire on diminue la température et on reprend le processus de calcul de l'énergie. S'il s'agit d'un problème de maximisation, il suffit de prendre l'opposé de l'énergie ($-\Delta E_{ij} = E_i - E_j$) comme fonction objectif. L'algorithme du recuit simulé pour un problème de minimisation est donné à la figure (1.5).

- 1) Choisir une solution initiale $s \in S$
- 2) $T \leftarrow T_0$ {initialiser la température courante} ;
- 3) Répéter
- 4) Choisir aléatoirement $s' \in V(s)$
- 5) Calculer $\Delta \leftarrow f(s') - f(s)$;
- 6) Si $((\Delta < 0))$ alors
- 7) $s \leftarrow s'$;
- 8) Sinon r nombre aléatoire de $[0,1]$;
- 9) Si $(r < \exp(\Delta/T))$ alors
- 10) $s \leftarrow s'$;
- 11) $T \leftarrow 0.9 * T$ {réduire la température courante} ;
 Fin si
- Fin si
- 12) Jusqu'à conditions d'arrêt satisfaites.

Fig.1.5 Algorithme du recuit simulé.

La méthode du recuit simulé donne une solution de bonne qualité. Elle est facile à programmer et applicable à tous les problèmes ayant la caractéristique itérative tout comme elle offre une souplesse d'emploi car de nouvelles contraintes peuvent être incorporées facilement dans le programme. Néanmoins, cette méthode présente des inconvénients qui apparaissent dans le nombre important de paramètres tels la température initiale, le taux de décroissance de la température, la durée du palier de température, le critère d'arrêt, etc. ainsi que le temps de calcul qui est excessif dans certaines applications.

Dans la **recherche tabou** développée par Glover [7], Le principe est de passer à la meilleure solution voisine de la solution initiale à chaque itération même si celle-ci est plus mauvaise que la solution initiale. Ce critère autorise la dégradation de la fonction objectif et évite ainsi le blocage dans un minimum local. Le déplacement s'effectue de solution en solution en s'interdisant de revenir à une solution déjà rencontrée. Au cours de la recherche, ces solutions visitées sont mises dans une liste interdite appelée liste taboue. Cette liste de longueur finie k mémorise les k dernières solutions rencontrées ou les mouvements donnant ces solutions. Durant un nombre d'itération égal à k , il est interdit d'effectuer une transformation inverse à une transformation récemment effectuée c'est-à-dire ajouter une solution récemment supprimée ou supprimer une solution récemment ajoutée. Certaines

interdictions sont parfois inutiles. Par exemple, un mouvement qui mène à une solution meilleure que toutes celles visitées n'a aucune raison d'être interdit. Dans ce cas, le critère d'aspiration est introduit ; il s'agit de lever le statut *tabou* et d'exécuter cette solution. L'algorithme général de la recherche *tabou* pour un problème de minimisation est illustré sur la figure suivante (fig.1.6).

La recherche *tabou* présente l'avantage de posséder une mémoire sous la forme de la liste *tabou*. Cette mémoire peut être à court terme si on

Soit $N(s)$ un ensemble de voisinage de s et soit $N_T(s)$ tel que $N_T(s) = \{s' \in N(s) \text{ tel que } s' \notin \text{Tou } f(s') < f(s)\}$ et T la liste *tabou*.

- 1) Poser $T \leftarrow \emptyset$ {initialiser une liste taboue vide}
- 2) Générer une solution initiale s
- 3) Mettre $s^* \leftarrow s$ { s^* est la meilleure solution}
- 4) Répéter
- 5) Générer $N(s)$
- 6) Choisir s' qui minimise $f(s')$ dans $N(s)$
- 7) Si $f(s') < f(s^*)$ alors
- 8) Poser $s^* \leftarrow s'$ {condition d'aspiration : si la solution courante améliore la meilleure solution ; accepter la même si elle est taboue}
- 9) Poser $s \leftarrow s'$ {mettre à jour la solution courante}
- 10) $T \leftarrow T + s'$ {mettre à jour la liste tabou}
- 11) sinon si $s' \in N_T(s)$ alors
- 12) Poser $s \leftarrow s'$ {mettre à jour la solution courante si elle n'est pas taboue}
- 13) Si $f(s') < f(s^*)$ alors
- 14) Poser $s^* \leftarrow s'$
- 15) $T \leftarrow T + s'$
- Fin si

Fig.1.6 Algorithme de la recherche *tabou*.

mémorise la solution telle qu'elle est et à long terme si on mémorise la fréquence d'apparition au lieu d'interdire complètement le mouvement introduisant cette solution. Ainsi, les paramètres à fixer sont moins nombreux. Il s'agit de la taille de la liste *tabou* qui peut être statique déterminée à l'avance et constante durant tout le processus de résolution ou dynamique variante durant la résolution et du nombre total d'itérations. Cependant, dans plusieurs cas d'optimisation, la recherche *tabou* demande de lui ajouter les mécanismes d'intensification et de diversification qui introduisent de nouveaux paramètres à régler.

La **méthode GRASP** (Greedy Randomised Adaptive Search Procedure) qui signifie procédure de recherche adaptative aléatoire gloutonne est une métaheuristique introduite par FEO et RESENDE en 1995 [8]. Cette méthode répète un processus composé de deux étapes à chaque itération : la construction d'une solution par une méthode gloutonne et l'amélioration de cette solution par une méthode de recherche locale. Durant l'étape de construction, on choisit toutes les solutions réalisables, on prend les meilleures selon la fonction objectif puis on tire au hasard pour former la liste des candidats restreinte (LCR). Cette liste est mise à jour après chaque itération de construction ce qui donne l'aspect adaptatif de l'algorithme. On sélectionne au hasard une solution dans LCR qui sera améliorée en lui appliquant une méthode de recherche locale. On répète itérativement ces étapes jusqu'à l'obtention de la meilleure solution. L'algorithme de GRASP pour un problème de minimisation est le suivant (fig.1.7).

```
T = dimension du problème ;
Pour k=1 jusqu'à itération_max faire
Construction :
Solution=0 ;
Evaluer le cout incrémental des éléments candidats ;
Tant que Dimension (solution) ≠ T : Construire LCR : la liste de candidats restreinte ;
Sélectionner au hasard une solution s de LCR : solution = Ajouter (solution, s)
Fin
Recherche locale :
Tant que solution non optimale localement :
Trouver s' ∈ Voisinage (solution) tel que  $f(s') < f(solution)$ 
    solution = s'
Fin
meilleure_solution = min (solution, meilleure_solution) ;
Fin
Retourner meilleure_solution
```

Fig.1.7 Algorithme de la méthode GRASP.

L'aspect le plus intéressant de GRASP est sa facilité d'implémentation ainsi que le nombre restreint de paramètres à régler ; il suffit de fixer la longueur de la liste LCR et le nombre maximum d'itérations. Cependant, cette méthode est moins performante par rapport aux

autres métaheuristiques du fait de l'absence de mémoire ce qui implique l'obtention de la même solution en plusieurs reprises.

Contrairement aux méthodes précédentes, **les méthodes de recherche à base de population** se basent sur une population d'individus qui interagissent entre eux afin de s'améliorer ou de produire de nouveaux individus plus performants. Elles travaillent sur un ensemble de points de l'espace de recherche. Ces métaheuristiques sont inspirées de la biologie et utilisant des phénomènes d'auto organisation. Parmi ces algorithmes à population on trouve, particulièrement, les algorithmes évolutionnaires comme les Algorithmes Génétiques, les colonies de fourmis, les essaims de particules etc.

Les **algorithmes génétiques** s'inspirent des principes de la génétique. Pour cette raison, ces algorithmes accordent une grande importance à la distinction entre la représentation génétique d'un individu (*génotype*) et sa représentation réelle (*phénotype*). L'opérateur principal utilisé par les algorithmes génétiques pour la construction de nouvelles solutions est l'opérateur de *recombinaison* appelé aussi *croisement*. Cet opérateur récupère des parties du génotype de deux ou plusieurs solutions ou parents qu'il combine pour construire un ou plusieurs nouveaux génotypes ou enfants, héritant ainsi de certaines de leurs caractéristiques. L'utilisation de la recombinaison, seule, ne permet pas d'introduire du matériel génétique nouveau puisque cet opérateur combine le matériel déjà présent dans la population. Pour remédier à ce problème, les algorithmes génétiques utilisent *la mutation*, comme opérateur secondaire permettant d'introduire de nouveaux gènes inexistants dans la population. Le principe général d'un algorithme génétique est présenté dans la figure 1.8 (fig.1.8).

```
1) Initialisation :  
   K :=nombre total de générations ;  
   N :=taille de la population ;  
   Pc :=Probabilité de croisement ;  
   Pm :=probabilité de mutation ;  
   POP :=population courante ;  
   PINT :=population intermédiaire contenant N éléments chacun ;  
2) Répéter  
   Pour i=0 jusqu'à K  
     Évaluer [POP]  
     Sélection [PINT, POP]  
     Croisement [PINT, Pc]  
     Mutation [PINT, Pm]  
3) Remplacement POP=PINT  
   Évaluer [POP]  
Solution meilleur=meilleur [POP]
```

Fig.1.8 Principe général d'un algorithme génétique.

L'algorithme de **l'Evolution Différentielle (ED)** (Differential Evolution) a été développé par Price et Storn en 1995 [9] pour résoudre les problèmes d'optimisation combinatoires. Cet algorithme utilise les mêmes opérateurs évolutionnaires que l'algorithme génétique (sélection, croisement et mutation) mais le fonctionnement est différent (fig.1.9). En effet, cette méthode crée de nouveaux individus. Une nouvelle solution est créée à partir des différences entre deux individus de la population. Si on considère un problème à D dimensions avec une population de N individus évoluant à chaque génération t , l'algorithme de la méthode ED pour un problème de minimisation est donné à la figure 1.8. Dans cet algorithme F est un nombre réel appartenant à l'intervalle $[0,2]$ appelé facteur d'amplification. De nombreuses améliorations ont été proposées pour le choix des paramètres. Cette métaheuristique a été utilisée avec succès dans le cas des problèmes réels notamment pour résoudre des problèmes continus.

```

1) Générer une population initiale
2) Pour t=0,...,T
3) Pour i=0,...,N
4) Générer aléatoirement r1, r2 et r3 avec r1≠r2≠r3
   Choisir aléatoirement trois individus xr1,t, xr2,t, xr3,t
   Mutation :
     Créer un mutant vi,t à partir de l'individu xi,t et de xr1,t, xr2,t, xr3,t
     vi,t+1 = xr1,t + F*( xr2,t - xr3,t)
   Croisement :
     Créer l'individu test ui,t à partir du mutant vi,t et de xi,t
   Sélection :
     Si f( ui,t ) < f( xi,t ) alors xi,t ← ui,t
     Sinon xi,t+1 ← xi,t
5) Fin pour
6) Fin pour

```

Fig 1.9 Algorithme de l'évolution différentielle.

L'algorithme **des colonies de fourmis** (Ant Colony Optimisation) est un algorithme d'intelligence en essaim dont le principe est basé sur la manière dont les fourmis cherchent leurs nourritures et retrouvent leur chemin pour retourner dans la fourmilière (fig.1.10). Initialement, les fourmis explorent les environs de leur nid de manière aléatoire [10]. Sitôt qu'une source de nourriture est repérée par une fourmi, son intérêt est évalué (quantité et qualité) et la fourmi ramène un peu de nourriture au nid. Les fourmis peuvent déposer des phéromones au sol, grâce à une glande située dans leur abdomen et former, ainsi, des pistes odorantes qui pourront être suivies par leurs congénères. Les traces laissées s'accumulent au fur et à mesure que la piste est rejointe par plus de congénères. Les phéromones ont comme caractéristique l'évaporation en fonction du temps. Les pistes les plus longues seront donc abandonnées au profit de la plus courte. Ainsi, pour illustrer le principe de cette méthode posons A un ensemble de K fourmis. Une fourmi de cet ensemble va démarrer de la ville i à l'instant t vers la ville j , la destination est choisie en fonction de la visibilité η_{ij} (l'inverse de la distance) et de la quantité de phéromones $\tau_{ij}(t)$ déposée entre les deux villes.

Répéter :
 Pour $i=1$ à k faire
 Construire le Trajet (i)
 Fin Pour
 Mettre à Jour Phéromones ()
 Jusqu'à ce que le critère de terminaison soit satisfait.
 Dans la procédure *Construire le Trajet (i)*, chaque fourmi construit un trajet selon la probabilité $p^{k_{ij}}(t)$ tel que :

$$p^{k_{ij}}(t) = [\tau_{ij}(t)]^\alpha * [\eta_{ij}]^\beta / [\sum_{l \in J_{ik}} (\tau_{il}(t))^\alpha * (\eta_{il})^\beta] \quad \text{si } j \in J_{ik}$$

$$p^{k_{ij}}(t) = 0 \quad \text{sinon}$$

Fig 1.10 Algorithme de colonies de fourmis.

α et β sont deux paramètres contrôlant respectivement l'influence du taux de phéromone sur le trajet (ij) et l'influence de la distance sur le trajet (ij). La procédure *mettre à jour Phéromone()* consiste à mettre à jour le taux de phéromones τ pour la fourmi k en appliquant la formule suivante :

$$\tau_{ij}(t+1) = (1 - \rho) * \tau_{ij}(t) + \Delta\tau_{ij}(t) \quad (1.1)$$

avec $\Delta\tau_{ij}(t) = \sum_{x=1}^k (\Delta\tau_{ij}^x(t))$ et $\Delta\tau_{ij}^k(t) = \frac{Q}{Lk(t)}$

Q est une constante et $Lk(t)$ est la longueur totale de la tournée de la fourmi k à l'instant t . Pour éviter d'être piégé dans des optima locaux, les algorithmes de colonies de fourmis utilisent la stratégie d'évaporation des pistes de phéromones dont la solution est mauvaise [11]. Les algorithmes de colonies de fourmis sont très efficaces en optimisation combinatoire grâce à leur robustesse *ie.* la recherche reste efficace même si certains de ses individus sont défaillants, et à sa décentralisation, *ie.* les fourmis n'obéissent pas à une autorité centralisée, comme ils ont l'avantage d'être exécutés en parallèle. Néanmoins, cette méthode présente l'inconvénient de réglage d'un nombre important de paramètres (nombre de fourmis, visibilité, quantité de phéromones etc.).

L'optimisation **par les essaims de particules** (Particle Swarm Optimisation ou PSO) est issue d'une analogie avec les comportements collectifs de déplacements chez certains groupes d'animaux, comme

les bancs de poissons, les nuées des oiseaux, les araignées etc. [12]. En effet, tout comme ces animaux se déplacent en groupe pour trouver de la nourriture ou éviter les prédateurs, les algorithmes à essaim de particules recherchent des solutions pour un problème d'optimisation. L'essaim particulaire est constitué d'une population de particules. Chaque particule i correspond à une solution du problème traité. Elle est caractérisée par une position $\vec{x}_i$ et une vitesse (vélocité) de changement de position $\vec{v}_i$ et garde en mémoire sa meilleure position atteinte qui peut être parfois la position courante. À chaque itération t , la position et la vitesse sont mises à jour en tenant compte de la meilleure position $\vec{p}_i$ de la particule et de la meilleure position $\vec{p}_g$ de ses voisins suivant la formule citée ci-dessous :

$$\begin{cases} \vec{v}_i(t) = \omega * \vec{v}_i(t-1) + c1 * \rho1 * (\vec{p}_i - \vec{x}_i(t-1)) + c2 * \rho2 * (\vec{p}_g - \vec{x}_i(t-1)) \\ \vec{x}_i(t) = \vec{x}_i(t-1) + \vec{v}_i(t) \end{cases} \quad (1.2)$$

ω est une constant appelée coefficient d'inertie. Elle joue un rôle important dans la convergence. Une faible valeur de ω nous guide à une recherche locale tandis qu'une grande valeur nous permet une exploration globale de l'espace de recherche [13]. $c1$ représente l'attraction de la particule vers sa meilleure position ; c'est la mémoire propre à la particule, tandis que $c2$ représente l'attraction vers la meilleure de ses voisines engendrant la mémoire partagée. $\rho1$ et $\rho2$ sont des variables aléatoires tirés de l'intervalle $[0,1]$ à chaque itération. Eberhart et Shi [14] ont suggéré d'utiliser $c1=c2=2$.

La figure (1.11) montre le mécanisme de déplacement des essaims de particules.

Maurice et Clerc [15,16] ont introduit un nouveau paramètre appelé coefficient de constriction 'k' qui est calculé comme suit :

$$K = \frac{2}{|2 - \Phi - \sqrt{\Phi * \Phi - 4 * \Phi}|} \quad \text{avec } \Phi = c1 + c2 \text{ et } \Phi > 4$$

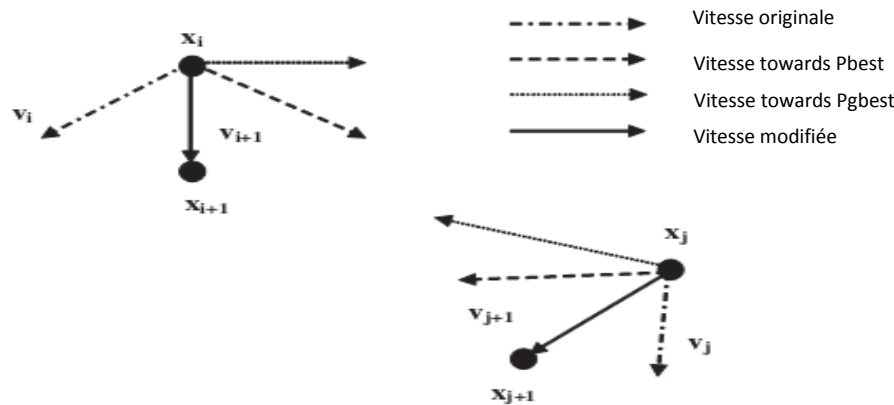


Fig.1.11 Mécanisme de déplacement des essaims de particules.

La formule de PSO devient :

$$\begin{cases} \vec{v}_i(t) = k * (\vec{v}_i(t-1) + c1 * \rho1 * (\vec{p}_i - \vec{x}_i(t-1)) + c2 * \rho2 * (\vec{p}_g - \vec{x}_i(t-1))) \\ \vec{x}_i(t) = \vec{x}_i(t-1) + \vec{v}_i(t) \end{cases} \quad (1.3)$$

Ce coefficient a le même rôle que le coefficient d'inertie et quand il est utilisé, Φ est fixé à 4.1 et $c1=c2=2.05$. Les étapes principales de l'algorithme OEP sont présentées sur la figure (1.12).

- 1) N := nombre d'individus ;
- 2) $\vec{x}_i$:= position de la particule P_i ;
- 3) $\vec{v}_i$:= vitesse de la particule P_i ;
- 4) P_{best_i} := meilleure fitness obtenue pour la particule P_i ;
- 5) $\vec{x}_{P_{best_i}}$:= position de la particule P_i pour la meilleure fitness ;
- 6) $\vec{x}_{g_{best}}$:= position de la particule ayant la meilleure fitness de tout le voisinage ;
- 7) Initialiser aléatoirement la population ;
- 8) Répéter
- 9) Pour i de 1 à N faire
- 10) Si $(f(\vec{x}_i) < P_{best_i})$ alors $f(\vec{x}_i) = P_{best_i}$ et $\vec{x}_{P_{best_i}} = \vec{x}_i$
- 11) Fin Si
- 12) Fin Pour
- 13) Pour i de 1 à N faire

$$\vec{v}_i(t) = \omega * \vec{v}_i(t-1) + c1 * \rho1 * (\vec{p}_i - \vec{x}_i(t-1)) + c2 * \rho2 * (\vec{p}_g - \vec{x}_i(t-1))$$

$$\vec{x}_i(t) = \vec{x}_i(t-1) + \vec{v}_i(t)$$
- 14) Fin Pour
- 15) Jusqu'à ce que le critère d'arrêt soit atteint.

Fig.1.12 Algorithme des essaims de particules.

La méthode des essaims particulaires est très puissante en optimisation combinatoire. Elle a rencontré un vif succès depuis sa création et ça grâce à sa simplicité et son efficacité. Notons qu'il existe

d'autres métaheuristiques qui sont moins populaires et peu efficaces. Parmi ces méthodes, on cite quelques variantes du recuit simulé, les algorithmes à estimation de distribution, les systèmes immunitaires artificiels, les algorithmes inspirés des insectes sociaux, la méthode du bruitage, etc. Une étude plus détaillée se trouve dans l'ouvrage de Johann Dréo et al.[4].

3. Conclusion

Dans ce chapitre, nous avons présenté quelques méthodes d'optimisation combinatoires en s'appuyant sur les caractéristiques principales des métaheuristiques. Ces dernières sont très efficaces en optimisation difficile sans avoir besoin de modifier la structure de base de l'algorithme utilisé. Elles sont devenues très populaires grâce à leur simplicité d'emploi dans différents domaines. Il est à noter qu'une bonne performance nécessite souvent une formalisation adéquate du problème posé et une adaptation intelligente d'une métaheuristique. Malgré le succès remarquable de leur démarche, les métaheuristiques présentent des difficultés à lesquelles est confronté l'utilisateur dans le cas d'un problème concret comme le choix d'une méthode efficace pour avoir une solution optimale et le réglage des paramètres qui peut être réalisable en théorie mais irréalisable en pratique. Les chercheurs visent à surpasser ces difficultés en proposant des techniques d'amélioration dont on cite l'hybridation des métaheuristiques. Cette hybridation exploite la puissance de plusieurs algorithmes, et les combine en un seul méta-algorithme. Le chapitre suivant sera consacré à l'étude de l'hybridation des métaheuristiques.

Chapitre 2

Hybridation des métaheuristiques

1. Introduction

La famille des métaheuristiques regroupe un ensemble de méthodes performantes en optimisation difficile. Elles offrent des solutions approchées de bonne qualité pour des applications réelles de grande taille. Ces méthodes sont généralement classées en deux grandes catégories : les méthodes à la recherche locale qui se basent sur la stratégie d'intensification et les méthodes à la recherche globale qui se basent sur la diversification et pour avoir un résultat pertinent, il faut toujours essayer de réaliser un équilibre entre ces deux stratégies. Quelque soit la méthode utilisée, elle présente des avantages et des inconvénients. Des essais d'amélioration sont proposés pour obtenir la méthode la plus efficace possible. L'hybridation des méthodes peut permettre de trouver une amélioration car les avantages et les inconvénients de chaque méthode se compensent.

2. Notion d'hybridation

L'hybridation consiste à combiner les caractéristiques de deux méthodes différentes pour tirer les avantages des deux méthodes [17]. Les origines des algorithmes hybrides des métaheuristiques reviennent aux travaux de Glover [18], J. J. Grefenstette [19] et Mühlenbein et al. [20]. Chacun d'eux a introduit une méthode de descente simple pour améliorer une recherche évolutive. Mais à cette période, la plupart des chercheurs n'y accordait que peu d'intérêts. Actuellement, les métaheuristiques hybrides sont devenues plus populaires car les meilleurs résultats trouvés pour plusieurs problèmes d'optimisation combinatoires ont été obtenus avec des algorithmes hybrides. L'hybridation des métaheuristiques peut être divisée en deux grandes parties: hybridation des métaheuristiques avec des métaheuristiques et hybridation des métaheuristiques avec des méthodes exactes.

3. Hybridation métaheuristiques/métaheuristiques

Toutes les métaheuristiques que nous avons étudiées précédemment ont été hybridées avec succès dans plusieurs applications. Selon la taxonomie proposée dans [17] l'hybridation des métaheuristiques entre elles se fait en deux classifications principales: une classification hiérarchique et une classification à plat (fig.2.1).

3.1 Classification hiérarchique des métaheuristiques

Cette classification est caractérisée par le niveau et le mode de l'hybridation. Le niveau d'hybridation peut être bas (Low-Level) ou haut (High-Level). Dans le niveau bas, une métaheuristique remplace un opérateur d'une autre méthode qui l'englobe. Par contre, dans le niveau haut de l'hybridation, chaque métaheuristique garde sa propriété au cours de l'hybridation. Chaque niveau d'hybridation engendre deux modes de coopération à savoir, le mode relais et mode

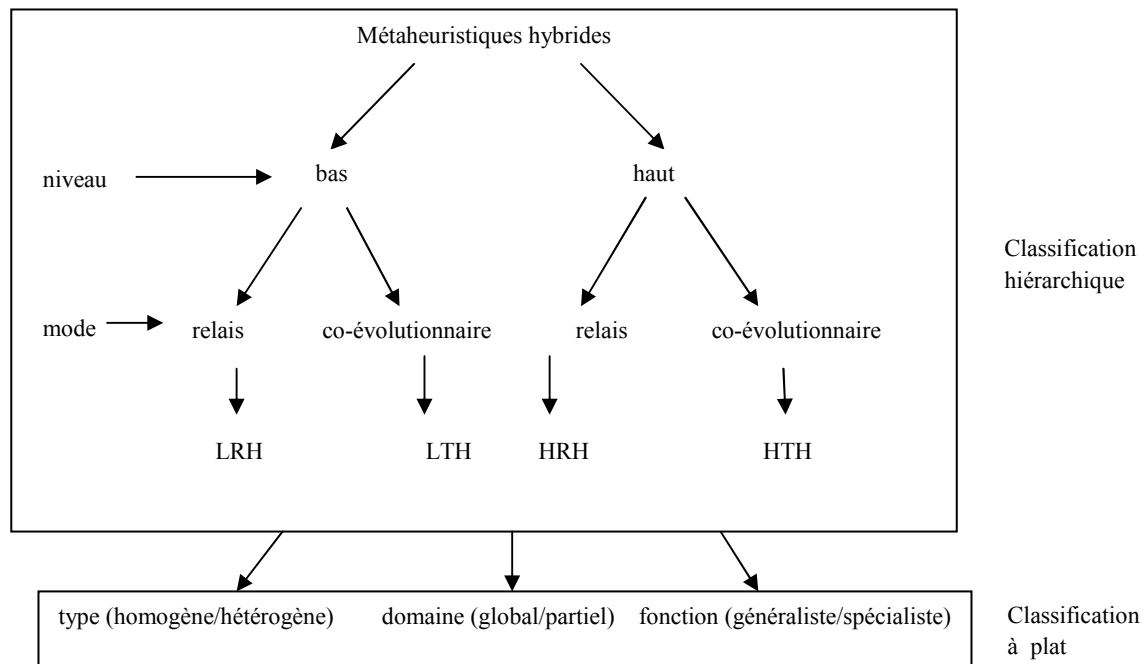


Fig.2.1 Taxonomie de l'hybridation des métaheuristiques

co-évolutionnaire. Dans le mode relais, les méthodes sont exécutées séquentiellement, c'est-à-dire le résultat de la première méthode est le début de la méthode suivante. Quand les différentes méthodes fonctionnent en parallèle pour explorer l'espace de recherche, on parle de mode co-évolutionnaire. La combinaison des modes et des niveaux donne quatre classes d'hybridation qui sont l'hybridation relais de bas niveau, l'hybridation co-évolutionnaire de bas niveau, l'hybridation relais de haut niveau et l'hybridation co-évolutionnaire de haut niveau.

L'hybridation relais de bas niveau (Low-level Relay Hybrid) englobe les métaheuristiques à base de solution unique dans lesquelles une autre méthode est incorporée pour former un nouvel algorithme. Pour résoudre le problème du voyageur de commerce (Traveling Salesman Problem) et le problème de la partition de graphe, Martin et Otto [21] ont inséré la méthode de descente dans un algorithme de recuit simulé.

L'hybridation co-évolutionnaire de bas niveau (Low-level Teamwork Hybrid) consiste à incorporer une ou plusieurs métaheuristiques à

base de solution unique dans une métaheuristique à population de solutions. L'avantage de ce type d'hybridation est de compenser la puissance d'exploitation d'une recherche locale et celle d'exploration d'une recherche globale. Stützle et Hoos [22] incorporent une fonction de recherche locale dans un algorithme de colonie de fourmis pour résoudre le problème du voyageur de commerce et celui de partition de graphes. Gambardella, Taillard et *al.* [23] ont utilisé un algorithme de descente dans un algorithme de colonie de fourmis pour résoudre le problème de partition de graphes et McKendall et Shang [24] ont amélioré la méthode proposée par Gambardella, Taillard et *al.* en remplaçant l'algorithme de descente par l'algorithme du recuit simulé. Azimi [25] dans sa recherche de résolution du problème d'agencement d'horaire d'examens a proposé une amélioration de la solution de colonie de fourmis pour chaque cycle en appliquant le recuit simulé. Dans un algorithme génétique, J. Suh et D. van Gucht [26] ont remplacé l'opérateur de mutation par une recherche de descente et C. Fleurent et Ferland [27] ont appliqué cette classe d'hybridation pour résoudre le problème de partition de graphe en remplaçant l'opérateur de mutation d'un algorithme génétique par une recherche tabou.

L'hybridation relais de haut niveau (High-level Relay Hybrid) a lieu lorsque les métaheuristiques sont utilisées de manière séquentielle c'est-à-dire la (ou les) solution(s) finale(s) de la première métaheuristique est la (ou les) solution (s) initiale(s) de la métaheuristique suivante. Dans cette procédure, toutes les méthodes gardent leur intégrité (fig. 2.2).

Talbi et *al.* [28] ont introduit la recherche tabou à la fin d'un algorithme génétique pour améliorer les solutions obtenues. Comme il est connu d'utiliser une recherche locale pour initialiser une autre métaheuristique, Azimi a utilisé deux hybridations de type relais de

haut niveau pour résoudre le problème d'agencement d'horaires d'examens.

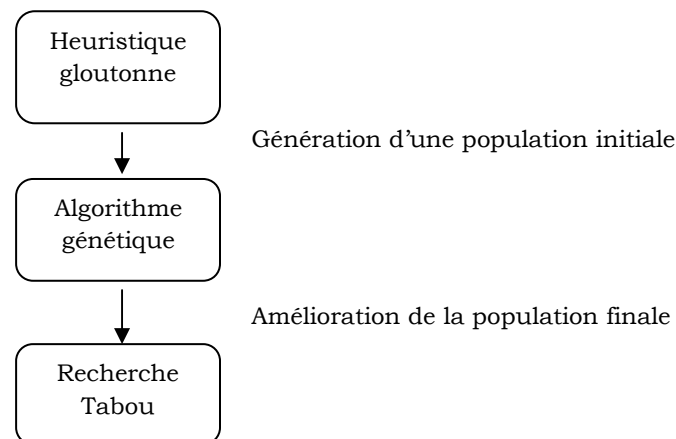


Fig.2.2 Exemple d'une hybridation relais de haut niveau [17].

Premièrement, il a initialisé les phéromones des colonies de fourmis par la recherche tabou puis il a pris la solution trouvée par les algorithmes de colonie de fourmis comme une solution initiale d'une recherche tabou. Pour résoudre le problème d'assignation quadratique, Lin, Kao et al. dans [29], proposent une hybridation où la méthode du recuit simulé crée une population initiale pour un algorithme génétique.

Dans le cas de **l'hybridation co-évolutionnaire de haut niveau** (High-level Teamwork Hybrid), les métaheuristiques utilisées travaillent en parallèle en échangeant des informations entre elles afin de trouver la solution optimale du problème posé. L'exemple le plus populaire est celui de l'algorithme génétique basé sur le modèle insulaire proposé par Tanese [30] (fig.2.3). Pour cette hybridation, la population est divisée en sous-populations réparties sur les sommets d'un hypercube dont lesquels un algorithme génétique est lancé. Chaque sommet correspond à une zone de recherche de solutions et périodiquement, des individus migrent entre les sommets en contribuant à trouver les solutions optimales

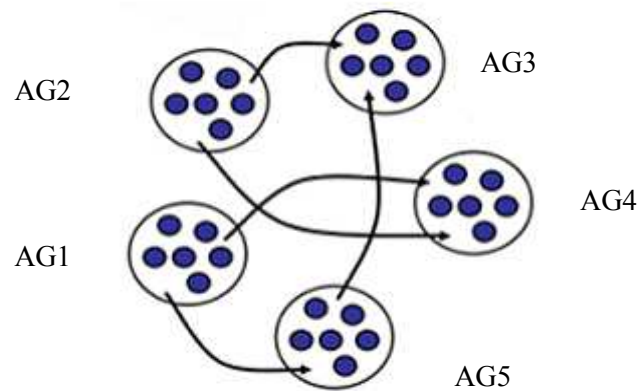


Fig.2.3 Le modèle insulaire d'un algorithme génétique [17].

Dans [31], Ghédira et Hammami ont fait appel à cette hybridation pour résoudre le problème de partitionnement de graphe. Ils ont utilisé la recherche tabou et le recuit simulé de telle sorte qu'à chaque intervalle régulier, les deux méthodes échangent d'informations pour trouver la meilleure solution. Notons qu'il existe d'autres travaux basés sur cette hybridation en utilisant des recuits simulés [32], des recherches tabou[33], la programmation génétique[34] et la stratégie évolutionnaire [35].

Dans tous ces travaux, il a été démontré [36] que les résultats ne sont plus performants si les métaheuristiques n'échangent pas d'informations.

3.2 Classification à plat des métaheuristiques

La classification à plat des métaheuristiques est caractérisée par le type des méthodes hybridées, leur domaine d'application et la nature de leurs fonctions. Selon le type d'hybridation, on trouve des méthodes hybridées homogènes où les algorithmes utilisés se basent sur la même métaheuristique comme le modèle insulaire et des méthodes hybridées hétérogènes où les métaheuristiques utilisées sont différentes. Le modèle proposé dans [31] est une hybridation de haut niveau co-évolutionnaire hétérogène. Le domaine d'application des métaheuristiques hybridées permet de distinguer deux grandes classes

d'hybridation, les hybridations globales et les hybridations partielles. L'hybridation globale a lieu lorsque toutes les méthodes hybridées sont appliquées à la totalité de l'espace de recherche. Toutes les méthodes que nous avons étudiées précédemment sont des hybridations globales. A l'opposé, l'hybridation partielle décompose un problème en sous-problèmes où chacun a son propre espace de recherche. Dans ce contexte, Taillard [37] propose une décomposition du problème de routage de véhicules en appliquant la recherche tabou. L'idée est de diviser l'ensemble des villes à visiter en secteurs indépendants dont chacun représente un espace de recherche. Dans un autre article, Taillard et Voss [38] ont suggéré une méthode pour créer des sous-problèmes à partir du problème initial et le résoudre à l'aide d'une métaheuristique ou d'une méthode exacte. Selon le problème traité, Talbi distingue deux types d'hybridation, une hybridation généraliste et une hybridation spécialiste. On parle d'hybridation généraliste quand toutes les métaheuristiques hybridées traitent le même problème d'optimisation. Toutes les hybridations que nous avons citées dans la classification hiérarchique font partie de cette catégorie. A l'inverse, les hybridations spécialistes ont lieu lorsque chaque métaheuristique traite un problème différent. Un exemple de ce type est l'utilisation d'une métaheuristique pour initialiser les paramètres d'une autre métaheuristique. Krueger [39] optimise les paramètres d'un recuit simulé à l'aide d'un algorithme génétique. Abbattista [40] optimise ceux d'un algorithme de colonie de fourmis à l'aide d'un AG et Shahookar et Mazumder [41] optimisent les paramètres d'un algorithme génétique à l'aide d'un autre algorithme génétique.

4. Hybridation métaheuristiques/méthodes exactes

L'hybridation des métaheuristiques avec les méthodes exactes a été moins usuelle que l'hybridation métaheuristique/métaheuristique car la plupart des chercheurs la considérait assez inutile. En ces derniers

temps, cette hybridation commence à s'étendre et un grand nombre d'articles a été publié concernant cette étude. Dans [17], Talbi a généralisé sa taxonomie aux méthodes exactes de sorte que la classification hiérarchique est applicable à ce genre d'hybridation.

La classe d'hybridation relais de bas niveau (LRH) est plus efficace en ayant une métaheuristique hybridée avec une méthode exacte. Un exemple de ce type est proposé par Augerat et al. [42] en utilisant un algorithme de Branch & Cut et une recherche tabou pour résoudre le problème de tournée de véhicule avec contraintes de capacité.

La classe d'hybridation co-évolutionnaire de bas niveau (LTH) regroupe une métaheuristique à base de population dont laquelle un opérateur est remplacé par une méthode exacte. Cotta et al. [43] proposent une hybridation entre un algorithme génétique et la méthode exacte Branch & Bound qui remplace l'opérateur de recombinaison. Pour résoudre le problème du voyageur de commerce, Jahuira et al. [44] proposent des hybridations entre un algorithme génétique et une méthode exacte. Ici, l'opérateur de recombinaison est remplacé par un algorithme de Branch & Bound et la population initiale est choisie à l'aide d'un arbre de recouvrement minimal engendré d'une manière exacte. Une autre hybridation de cette classe a été proposée par Kostikas et Fragakis [45]. Il s'agit d'implanter un algorithme de programmation génétique dans un algorithme de Branch & Bound pour générer la sélection des nœuds à explorer.

La classe LTH concerne aussi toutes les métaheuristiques basées sur l'exploration du voisinage où une méthode exacte intervient pour trouver la (ou les) meilleure solution du voisinage. Bent et Van utilisent cette idée pour résoudre le problème de voyageur de commerce. Dans la classe d'hybridation relais de haut niveau (HRH), les métaheuristiques et les méthodes exactes hybridées sont réalisées séquentiellement en gardant leur propriété. Bent et Van Hentenryck [46] ont utilisé cette hybridation pour résoudre le problème de tourné

de véhicule en minimisant le nombre de véhicules par le recuit simulé puis en optimisant la fonction objectif de chaque tournée par une recherche exacte dans un voisinage. Un autre exemple est la résolution du problème Flow-Shop proposée par Portmann et al. [47].

L'hybridation co-évolutionnaire de haut niveau (HTH) est difficile à réaliser entre une méthode exacte et une métaheuristique car chaque approche résout un problème différent puis, un échange d'information est indispensable entre elles. Dans ce contexte, Chabrier et al. [48] ont hybridé une recherche locale avec un algorithme de Branch and Price pour résoudre le problème de tournées de véhicules. L'exécution des méthodes s'effectue en parallèle tout en gardant une communication entre les méthodes. Pour la classification à plat, Talbi considère que les mêmes étapes qu'en hybridation métaheuristique/métaheuristique sont applicables à l'hybridation métaheuristique/méthodes exactes.

Dans [49], Puchinger et Raid proposent une autre classification pour l'hybridation des métaheuristicques avec les méthodes exactes. Ils divisent les méthodes hybridées en deux grandes classes, les hybridations collaboratives et celle intégratives.

Dans **l'hybridation collaborative**, les deux approches communiquent entre elles en échangeant des informations séquentiellement ou parallèlement. L'hybridation séquentielle a lieu quand une méthode exacte initialise une métaheuristique ou vice-versa. Applegate, Bixby et al. [50] ont proposé une collaboration séquentielle pour résoudre le problème du voyageur de commerce en utilisant une recherche locale pour créer un ensemble de solutions puis un algorithme exact pour désigner la solution optimale. Dans [51], Klau, Ljubic et al. ont hybridé un algorithme mémétique avec la méthode Branch & Cut tel que le but de ce dernier est d'optimiser les solutions trouvées par l'algorithme mémétique. Felzl et Raidi [52] ont utilisé la solution d'un programme linéaire d'une fonction pour sélectionner la population initiale d'un

algorithme génétique. Pour résoudre le problème d'ordonnancement de projet, Palpant, Artigues et al. [53] ont appliqué une métaheuristique pour construire la solution de départ puis un algorithme exact trouve une solution optimale. Applegate et Cook [54] ont utilisé cette stratégie pour résoudre les problèmes de Job-Shop. Quant à l'hybridation collaborative parallèle, elle a lieu quand la communication entre les métaheuristiques et les méthodes exactes s'effectue parallèlement. Pessan et al. [55] présentent une méthode d'hybridation d'un algorithme génétique avec la méthode Branch and Bound. L'idée est d'accélérer la résolution d'un problème par Branch and Bound et de réduire l'espace de recherche par l'algorithme génétique. La méthode la plus célèbre de cette classe est celle créée par Talukdar, Baerentzen et al. [56] pour résoudre des problèmes de type asynchrone ou Asynchronous Teams (A-Teams). Cette méthode contient un ensemble d'agents et de mémoires connectées à un réseau dirigé fortement cyclique. Chacun des agents est un algorithme d'optimisation approché ou exact qui peut travailler sur le problème à résoudre. Une population d'individus est contenue dans la mémoire et les différents agents peuvent ajouter, détruire ou modifier des individus. Plusieurs travaux d'hybridations collaboratives parallèles sont issus de ce principe. Souza [57] a employé cette stratégie pour résoudre le problème du voyageur de commerce. Chen, Talukdar et al. [58] l'ont utilisée pour le problème de planification d'atelier et Tsen [59] pour le problème de planification d'horaires.

Dans **l'hybridation intégrative**, l'un des algorithmes (métaheuristique ou exact) est intégré dans l'autre c'est-à-dire qu'une métaheuristique est intégrée dans une méthode exacte ou inversement. Pour résoudre un problème combinatoire de façon exacte, une méthode exacte est incorporée dans une métaheuristique. Burke, Cowling et al., dans [60], proposent une méthode de recherche locale incorporant un algorithme

exact pour résoudre le problème du voyageur de commerce. Raidl [61] et Chu et Beasley [62] présentent un algorithme génétique dont une méthode exacte est intégrée pour créer la population initiale ainsi que pour réparer les solutions non-réalisables. Congram, Potts et al., dans [63], proposent une méthode qui utilise la programmation dynamique dans un algorithme de recherche locale. Cotta et Troya [64] intègrent Branch and Bound dans un algorithme évolutif pour trouver le meilleur croisement possible entre deux individus sans avoir besoin d'appliquer une mutation, comme on trouve cette intégration pour décoder les chromosomes de certains algorithmes. Inversement, on peut intégrer une métaheuristique dans une méthode exacte. Khichane et al., dans [65], remplace la procédure de recherche arborescente par un algorithme de colonies de fourmis où chaque fourmi satisfait les contraintes du problème posé. Woodruff [66] dans son article intègre la recherche tabou dans Branch and Bound à chaque nœud de l'arbre pour trouver les meilleures solutions. Pour résoudre le problème de satisfiabilité maximale, French, Robinson et al.[67] ont proposé d'hybrider un algorithme génétique avec Branch and Bound. Au début, ils utilisent Branch and Bound pour créer la population initiale de l'algorithme génétique jusqu'à atteindre un certain critère et, à ce moment, l'algorithme génétique est lancé. Une fois terminé, les solutions trouvées sont insérées dans l'arbre de Branch and Bound qui reprend son exécution.

Une autre hybridation entre les métaheuristiques et les méthodes exactes a été récemment proposée par Stützle et Dumitrescu [68]. Cette classification n'est pas de type taxinomie. Elle fait la séparation selon l'objectif de la méthode utilisée pour l'hybridation. Dimistrescu et Stützle proposent la classification suivante :

- Utilisation d'une méthode exacte dans une recherche locale pour une exploration intensive du voisinage.

- Utilisation des solutions approchées de bonne qualité pour réduire l'espace de recherche d'une méthode exacte.
- Utilisation des bornes d'une méthode exacte pour une métaheuristique à population.
- Utilisation des informations récoltées par une méthode exacte pour orienter un algorithme de recherche locale ou globale.
- Utilisation d'une méthode exacte pour une fonction spécifique de la métaheuristique.

5. Discussion sur les classifications de méthodes hybrides

La taxonomie proposée par Talbi est la plus générale en regroupant des hybridations métaheuristiques/métaheuristiques et métaheuristiques / méthodes exactes, comme elle permet d'inclure facilement des nouveaux schémas d'hybridation. La classification de Stützle et Dumitrescu examinent l'hybridation des méthodes exactes avec les métaheuristiques mais tous les cas étudiés n'utilisent, généralement, que les métaheuristiques à recherche locale. La classification de Puchinger est plus large que cette dernière car elle englobe tous les types des métaheuristiques et malgré son orientation vers l'hybridation entre métaheuristiques et méthodes exactes. La stratégie de Puchinger ressemble beaucoup à celle proposée par Talbi et les deux classifications suivent les mêmes classes d'hybridation. Pour intégrer un algorithme dans un autre algorithme, Talbi applique l'hybridation de bas niveau (relais ou co-évolutionnaire) tandis que Puchinger applique l'hybridation intégrative. Pour une exécution séquentielle des méthodes, Talbi fait appel à l'hybridation relais de haut niveau et Puchinger, à l'hybridation collaborative séquentielle. Dans une exécution parallèle des algorithmes, on trouve l'hybridation de haut niveau co-évolutionnaire chez Talbi et l'hybridation

collaborative parallèle chez Puchinger. Talbi approfondit sa recherche en étudiant la nature homogène/hétérogène, globale/partielle, générale/spécialiste des différents algorithmes. Tous les travaux cités précédemment ont montré que les méthodes hybrides offrent de meilleurs résultats que les algorithmes non hybrides, autant pour les problèmes pratiques que théoriques [69].

6. Etude de l'hybridation des essaims de particules

L'évolution naturelle d'une population basée sur un algorithme de recherche peut être réalisée en intégrant des méthodes d'une grande efficacité pour résoudre des problèmes complexes. Les chercheurs ont amélioré les performances de l'algorithme des essaims de particules en lui incorporant d'autres approches d'optimisation. Dans ce qui suit, nous allons présenter quelques versions d'hybridations de l'algorithme des essaims particulaires (PSO) avec des méthodes d'optimisation.

6.1 Hybridation de PSO avec les algorithmes évolutionnaires

La première hybridation des essaims particulaires avec les algorithmes évolutionnaires a été développée par P. Angeline [70]. Elle a introduit l'opérateur de sélection dans l'algorithme de PSO pour choisir les bonnes particules qui vont subir une mutation et les mauvaises particules qui seront éliminées. Dans leur article, Miranda et al. [71] utilisent une hybridation entre les essaims particulaires et les stratégies évolutionnaires. Les paramètres $c1$ et $c2$, ainsi que pg , sont perturbés en appliquant une stratégie basée sur le processus de sélection. En 2002, Robinson et al. [72] ont proposé deux algorithmes hybrides de PSO avec les algorithmes génétiques (GA): GA-PSO et PSO-GA. Dans GA-PSO, l'algorithme génétique a été utilisé pour générer la population initiale de l'algorithme PSO tandis que, dans l'hybridation PSO-GA, PSO génère la population initiale de GA. Leur résultats ont démontré que l'hybridation PSO-GA est plus performante que GA-PSO.

PSO est favorable dans la phase de diversification alors que les algorithmes génétiques sont plus efficaces dans la phase d'intensification. Dans la même année, T.Krink et al. [73] ont hybridé PSO avec un algorithme génétique et la méthode de descente pour résoudre les problèmes d'optimisation globale sans contraintes. Chacune de ces méthodes peut être appliquée sur une sous-population d'individus où chaque individu est dynamiquement utilisé selon certaines règles. Grimaldi et al. [74] ont proposé une technique d'hybridation entre GA et PSO appelée Genetical Swarm Optimization (GSO) pour résoudre un problème d'optimisation électromagnétique. En GSO, la population est divisée en deux sous-populations qui seront évoluées suivant les étapes de GA et de PSO pour chaque itération. Ensuite, elles se regroupent pour actualiser la population initiale qui sera de nouveau divisée aléatoirement en deux sous-populations pour l'itération suivante pour une autre procédure de GA ou de PSO. Grimaldi et al. ont introduit un nouveau paramètre appelé constante d'hybridation qui indique le pourcentage de participation d'une population dans un l'algorithme génétique pour chaque itération. Si $HC=0$, la population est mise à jour suivant l'algorithme des essaims de particules et si $HC=1$, seulement l'algorithme génétique est appliqué. Dans le cas où $0 < HC < 1$, le pourcentage correspondant signifie que la population est développée par l'algorithme génétique et le reste par les essaims particulaires. L'application de GSO a été utilisée par Gandelli en 2005 dans [75] et en 2006 dans [76]. Settles et Soule [77] ont hybridé un algorithme génétique avec un algorithme des essaims particulaires en formant une hybridation appelée algorithme de reproduction en essaim (Breeding Swarm). Cet algorithme combine la règle de mise à jour de vitesse et de position de PSO avec l'opérateur de sélection, de croisement et de mutation de l'algorithme génétique. Dans un autre article, une autre hybridation entre PSO et GA est proposée par Kao et Zahara [78] pour résoudre un problème à

D-dimensions. Au début, on génère 4D individus aléatoirement qui sont des chromosomes dans le cas d'un algorithme génétique et des particules dans le cas des essaims particulaires. Ces 4D sont classés par la fitness du problème posé puis on applique les opérateurs de l'algorithme génétique sur les meilleurs 2D individus pour créer 2D nouveaux individus. A la fin, on applique la méthode de PSO sur le reste des individus non sélectionnés par la fitness. Le tableau 2.1 donne quelques hybridations entre PSO et GA.

Tableau 2.1 Hybridation entre l'algorithme des PSO et les AG

Auteurs	Année	Nom de l'hybridation
Robinson et al [72]	2002	GA-PSO et PSO-GA
Grimaldi et al [74]	2004	GSO
Juang [79]	2004	GAPSO
Settles et Soule [77]	2005	Reproduction en Essaim
Jiang et Chen [80]	2006	PSO-RDL
Kim [81]	2006	GA-PSO
Mohammadi et Jazaeri [82]	2007	PSO-GA
Kao et Zahara[78]	2008	GA-PSO
Ru et Jianhua[83]	2008	HGAPSO
Shanmugalatha et Slochanal [84]	2008	HPSO
Jeong et al [85]	2009	GA\PSO
Valdez et al [86]	2009	GA+PSO
Abdelkader [87]	2010	GAI-PSO

6.2 Hybridation de PSO avec l'algorithme de l'évolution différentielle

Dans cette partie, nous allons essayer de donner quelques hybridations effectuées entre les essaims de particules et l'évolution différentielle. Hendtlass [88] a introduit une hybridation de l'évolution différentielle (DE) pour sélectionner les meilleures positions de l'algorithme PSO. Cette hybridation est nommée SDEA. Kannan et al. [89] ont appliqué DE sur chaque particule de PSO dans un nombre d'itérations fini et ils ont remplacé la particule par le meilleur individu obtenu par DE. Talbi et Batauche [90] ont utilisé cette hybridation pour résoudre les problèmes de l'imagerie médicale ; ils ont appliqué les opérateurs de l'évolution différentielle uniquement sur les meilleures particules calculées par l'algorithme des essaims de

particules. Le tableau 2.2 présente quelques hybridations entre l'algorithme des essaims de particules et l'algorithme de l'évolution différentielle.

Tableau 2.2 Quelques hybridations entre l'algorithme des PSO et l'algorithme ED

Auteurs	Année	Nom de l'hybridation
Hendtlass [91]	2001	SDEA
Zang et Xie [92]	2003	DEPSO
Kannan et al [89]	2004	—
Talbi et Batauche [90]	2004	DEPSO
Hao et al [93]	2007	DEPSO
Jose et al [94]	2009	DEPSO
Xu et Gu [95]	2009	PSOPDE
Pant et al [96]	2009	DE-PSO
Khamsawang et al [97]	2010	PSO-DE

6.3 Hybridation de PSO avec d'autres algorithmes

Shelokar et al dans [98], ont proposé une hybridation entre PSO et les algorithmes de colonies de fourmis (ACO) appelée PSACO. Ici ACO est appliqué pour effectuer une recherche locale dans laquelle les fourmis sont guidées par la concentration des phéromones pour mettre à jour les positions calculées par les essaims particulaires. Ils ont appliqué cet algorithme pour résoudre des problèmes combinatoires sans contraintes. Dans [91], T. Hendtlass et Randall ont introduit ACO dans PSO de telle sorte qu'une liste des meilleures positions atteintes par les particules est enregistrée et le meilleur voisinage est sélectionné aléatoirement. Une hybridation de PSO avec la recherche tabou a été suggérée par Sha et Xsu [99] pour résoudre le problème Job Shop. He et Wang [100] ont hybridé PSO avec le recuit simulé pour résoudre les problèmes d'optimisation par contraintes. Le recuit simulé est appliqué sur les meilleures solutions de PSO pour s'échapper des optima locaux. Une liste d'hybridation de PSO avec d'autres algorithmes est présentée dans le tableau suivant.

Tableau 2.3 Quelques hybridations entre l'algorithme des PSO et d'autres algorithmes.

Auteurs	Année	Hybridation
Hendtllass et Randall [91]	2001	Essaims particulaires/colonies de fourmis
Wei et al. [101]	2002	Essaims particulaires /Programmation évolutionnaire
Sha et Hsu [99]	2006	Essaims particulaires/recherche tabou
Guo et al. [102]	2006	Essaims particulaires/méthode de descente
He et Wang [100]	2007	Essaims particulaires/recuit simulé
Mo et al. [103]	2007	Essaims particulaires/stratégie évolutionnaire
Shelokar [98]	2007	Essaims particulaires/colonies de fourmis
Firouzi et al. [104]	2008	Essaims particulaires/recuit simulé/K -means
Li et al. [105]	2008	Essaims particulaires/simplex
Ge et al. [106]	2008	Essaims particulaires/système humanitaire artificiel
Kavehand et Talatahary [107]	2009	Essaims particulaires/colonies de fourmis

Dans [108], R.Thangaraj et al. ont effectué une recherche exhaustive sur toutes les hybridations de l'algorithme des essaims particulaires. La figure 2.4 montre le nombre d'articles d'hybridation de PSO publiés de 2001 jusqu'à 2010 (Fig.2.4 et 2.5).

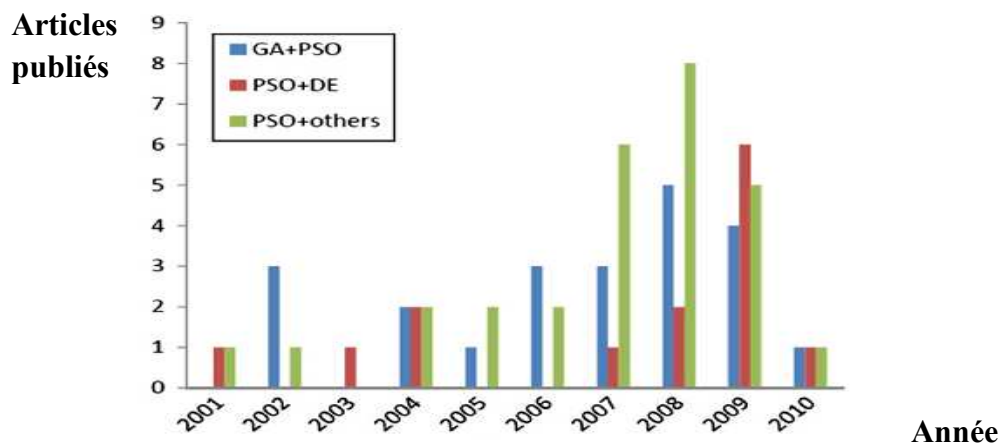


Fig.2.4 Histogramme montrant le nombre d'hybridation des essaims particulaires effectuée [108]

La figure. 2.5 nous donne le pourcentage des articles publiés de l'hybridation de PSO.

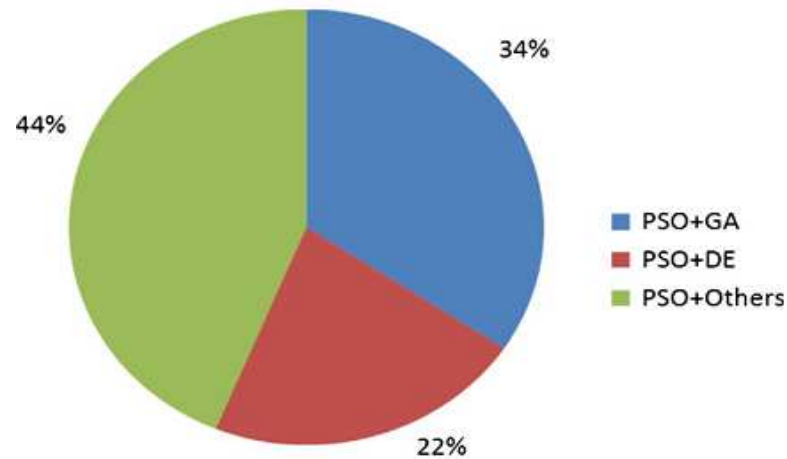


Fig.2.5 Graphique circulaire montrant la distribution des articles publiés de l'hybridation des essaims particulaires [108].

7 Conclusion

Dans ce chapitre, nous avons présenté les différentes techniques d'hybridation des métaheuristiques. Nous avons pu constater que la taxonomie proposée par Talbi est très simple et plus générale car elle englobe tous les types d'hybridations qui sont adaptables à un très grand nombre de problèmes d'optimisation combinatoires. A la fin, nous avons exposé quelques hybridations de l'algorithme des essaims de particules avec d'autres méthodes d'optimisation qui sont appliquées avec succès pour résoudre différents problèmes de différents domaines comprenant l'imagerie médicale, l'ingénierie, la programmation comme les algorithmes de l'ordonnancement, etc.

Dans le prochain chapitre, nous allons illustrer une hybridation entre les essaims de particules et le recuit simulé qu'on appliquera à la segmentation d'images.

Chapitre 3

Application d'une hybridation de deux métaheuristiques à la segmentation d'images.

1. Introduction

En traitement d'images, la segmentation est une opération importante pour l'extraction des informations qualitatives de l'image et autour de laquelle plusieurs travaux ont été proposés [109]. Elle consiste à partitionner l'image en régions distinctes et homogènes et ce, selon un ou plusieurs critères tels que le niveau de gris, la couleur et la texture ou tout autre critère spécifié par l'utilisateur. Il s'agit alors de trouver les frontières ou contours des régions selon un ensemble de critères préétablis de sorte que l'intersection entre ces régions constituent un ensemble vide et l'union des régions segmentées forme l'image source [110]. Les nombreuses méthodes de segmentation d'images proposées dans ce cadre peuvent être regroupées en différentes classes à savoir, la segmentation d'images par contours,

par régions, et par classification de pixels. Les méthodes de segmentation par contours sont basées sur la recherche des discontinuités locales présentes dans l'image alors que celles basées régions permettent de trouver des ensembles de pixels qui partagent des propriétés similaires. Ces deux méthodes peuvent toutefois être combinées en vue de l'amélioration des résultats de la segmentation. Cependant, aucune de ces méthodes n'offre réellement un résultat optimal. En effet, pour obtenir des régions très proches de la réalité du terrain, le problème de la segmentation d'images peut être posé en termes de problème d'optimisation combinatoire, difficile à résoudre. Ainsi, dans ce chapitre, nous proposons une segmentation d'images basée sur la méthode d'Otsu que nous allons, en premier lieu, intégrer dans le recuit simulé puis dans la méthode de PSO. En second lieu, nous procédons à l'hybridation de ces trois méthodes ensemble réunies. Pour évaluer la qualité de segmentation de ces méthodes, six images dont trois synthétiques et trois réelles mais complexes, sont utilisées.

2. Segmentation par classification des pixels

Il existe plusieurs méthodes de segmentation d'images. Celle basée sur la classification permet de fournir une partition de l'image en regroupant des pixels ayant des caractéristiques similaires dans une même classe. En effet, les régions sont définies par les ensembles de pixels connexes appartenant à la même classe. Lorsque des informations *a priori* sont utilisées pour la construction de classes sous la forme d'un ensemble d'apprentissage, la classification est dite supervisée [111]. Dans le cas où aucune connaissance *a priori* n'est disponible, on parle de classification non supervisée ou Clustering. Cette dernière catégorie qui consiste à trouver des groupes homogènes

au sein d'une population [112] a été utilisée dans de nombreux contextes et par de nombreux chercheurs dans différentes disciplines. Les principaux algorithmes rencontrés sont de la méthode des K-means [113], des méthodes hiérarchiques et de la méthode d'estimation-maximisation (EM)[114] entre autres. Ces algorithmes sont simples à implémenter et convergent rapidement vers des optima locaux [115]. Cependant, leur majeur inconvénient est qu'ils nécessitent de fournir, en entrée, une partition initiale de bonne qualité ainsi que le nombre possible de classes. Ces contraintes rendent l'utilisation de ces algorithmes peu intéressante dans le cas d'une segmentation automatique [116]. Pour définir les optima locaux en respectant ces contraintes, un problème de classification peut se ramener à un problème d'optimisation difficile [117]. Ainsi, plusieurs métaheuristiques ont été proposées dans le but d'obtenir une partition optimale des pixels et ce, en un temps raisonnable. Synder et al. dans [118], ont suggéré l'adaptation du recuit simulé aux problèmes de segmentation par l'approche paramétrique dont le but est d'approcher l'histogramme à l'aide d'une combinaison de gaussiennes en vue de trouver les paramètres. Pour leur part, L. Jourdan et al. dans [119], ont présenté une implémentation de l'algorithme du recuit simulé pour remédier aux problèmes rencontrés dans l'analyse de formes. Ensuite, en élaborant une comparaison des résultats avec ceux obtenus par l'algorithme des K-means, ils ont prouvé que le recuit simulé produit des résultats aussi bons que ceux fournis par les autres méthodes. Néanmoins, toutes ces nouvelles méthodes de classification présentent un temps de calcul élevé en plus de la difficulté de réglage des paramètres du recuit simulé.

Dans notre travail, nous allons considérer la caractéristique de classification des pixels selon leurs niveaux de gris en utilisant le

multiseuillage par l'analyse d'histogramme. En effet, dans le cas d'un histogramme à N classes, le seuillage consiste à déterminer $N-1$ seuils de façon à ce que chaque classe soit associée à un intervalle de niveaux de gris. Les seuils de segmentation correspondent, en général, aux minima des vallées présentes dans l'histogramme.

Notons que les méthodes de seuillage peuvent être divisées en deux grandes classes à savoir, les méthodes paramétriques et les méthodes non paramétriques.

Les méthodes paramétriques de seuillage supposent que les différentes classes de l'image suivent une densité de probabilité prédéfinie [119]. Dans ce cas, l'histogramme est approché par une combinaison linéaire de gaussienne et les seuils sont choisis aux intersections de celles-ci. Une telle opération nous permet d'obtenir une expression analytique approchée de l'histogramme et, donc, de pouvoir choisir les seuils optimaux plus facilement. Plusieurs travaux illustrant ces méthodes ont été publiés dont on cite Feng et al.[120], Gonzalez et al.[121], Sezgin et al.[122]... etc.

Les méthodes non paramétriques, pour leur part, permettent de trouver les seuils optimaux sans tenir compte de la forme de l'histogramme. Les méthodes de référence de cette catégorie sont la méthode d'Otsu [123] et la méthode de Kapur et al. [124] Dans notre application, nous avons utilisé la méthode d'Otsu pour segmenter des images à multiseuillage.

3. la méthode d'Otsu

Dans son article [123], Otsu a étudié trois critères pour calculer les seuils optimaux d'une image en utilisant la variance interclasse qui est la plus utilisée grâce à sa simplicité de mise en œuvre. La méthode nécessite une normalisation de l'histogramme pour qu'elle soit indépendante du nombre de pixels dans l'image. Ainsi, si l'on considère

N , le nombre total de pixels inclus dans l'image, H la hauteur de l'image et L la largeur alors $N=H*L$ et, si n_i est le nombre de pixels pour un niveau de gris i déterminé entre 0 et 255 on obtient, le nombre de pixels p_i pour un niveau de gris i en utilisant l'expression (3.1) suivante :

$$p_i = n_i / N \quad (3.1)$$

Dans le cas d'une binarisation, on distingue deux classes de pixels C_1 et C_2 telles que, pour C_1 , les niveaux de gris vont de 1 à t et pour C_2 , les niveaux de gris vont de $t+1$ jusqu'à L . Le seuil optimal t^* est celui qui maximise la variance interclasse suivante :

$$t^* = \text{Arg max } \sigma_B^2 \quad (3.2)$$

avec $\sigma_B^2 = P_t^*(P_t-1)^*(\mu_1-\mu_0)$ où $\mu_0 = \sum_{i=1}^t i.P_i/P_t$, $\mu_1 = \sum_{i=t+1}^{L-1} i.P_i/(1-P_i)$ et $P_t = \sum_{i=0}^t P_i$

Dans le cas de multiseuillage, la méthode d'Otsu peut être étendue au calcul d'un ensemble de seuils $T = (t_1, t_2, \dots, t_{m-1})$

$$T^* = \text{Arg max } \sigma_B^2, 1 \leq t_1 < \dots < t_{m-1} < L, \quad (3.3)$$

Cette méthode est appliquée dans plusieurs domaines, notamment en segmentation d'images [125][126].

Dans ce qui suit, nous allons combiner, d'abord séparément, les métaheuristiques RS et PSO avec la méthode d'Otsu. Ensuite nous procédons à l'hybridation de ces trois méthodes ensemble réunies. Pour évaluer la qualité de segmentation de ces méthodes, un critère basé sur la mesure du rapport signal sur bruit est utilisé. Ce rapport est défini par :

$$PSNR = 20 \log \left(\frac{255}{E} \right) \quad (3.4)$$

où E représente la racine carrée des erreurs quadratiques entre les pixels de l'image d'origine et les pixels de l'image segmentée

représentée en niveau de gris. L'expression de E est donnée par la formule suivante :

$$E = \sqrt{\sum_{i=1}^N \frac{(g_i - m_i)^2}{N}} \quad (3.5)$$

où N représente le nombre total des pixels de l'image. La segmentation est dite bonne si la valeur de PSNR diminue et vice versa. Pour chaque image segmentée, nous allons effectuer une comparaison entre les résultats de PSNR, les valeurs de la fonction objectif et le temps nécessaire pour l'exécution de la segmentation.

4. Intégration de la méthode d'Otsu dans l'algorithme RS

D'un point de vue théorique, le RS permet d'approcher de près la solution optimale plus rapidement qu'une exploration exhaustive dans l'espace de recherche [109]. Pour assurer la sélection des seuils optimums de segmentation en réduisant le temps d'exécution, nous combinons le RS avec la méthode d'Otsu. Cette combinaison nous permet d'effectuer un multi seuillage notamment lorsque le nombre de classe croît. La recherche de la solution optimale est basée sur la maximisation de la variance interclasse. L'algorithme suivant décrit le processus général de segmentation basé sur le RS en maximisant la variance interclasse d'Otsu (fig.3.1).

1. Initialisation de la température T
2. Fixer s initial d'après la fonction objectif d'Otsu
3. Générer aléatoirement une solution acceptable et calculer son énergie Δ .
4. Tant que le nombre de solutions accepté est non nul
 - 4.1 Répéter pour un nombre d'itérations donné :
 - 4.2 Générer aléatoirement une transition et calculer la variation d'énergie associée Δf . Si $\Delta f < 0$ alors s' est la meilleure solution sinon la solution est acceptée selon la probabilité $P = \exp(-\Delta f/T)$.
 - 4.3 Décroître T : $T = 0.9 * T$
5. Retourner le meilleur état rencontré au cours de la recherche.

Fig.3.1 Algorithme Otsu-RS de segmentation.

Les valeurs des paramètres de l'algorithme proposé ont été fixées de façon empirique et de manière à avoir le meilleur compromis entre la vitesse de convergence (temps) et la qualité de la segmentation.

5. Intégration de la méthode d'Otsu dans l'algorithme PSO

Comme nous l'avons déjà cité en chapitre précédent, cette métaheuristique est très efficace en optimisation combinatoire. Elle a été hybridée avec succès avec d'autres méthodes dans différents domaines de recherche. L'optimisation par PSO a été appliquée à la segmentation d'images grâce aux travaux de Du Feng et al. Dans ce qui suit, nous allons présenter un autre algorithme de segmentation basé sur l'introduction des PSO dans la méthode d'Otsu et, ceci, pour remédier au problème de la vitesse de convergence. En effet, la méthode d'Otsu présente son inefficacité quand le nombre de classes augmente car elle demande un temps d'exécution très élevé. Les paramètres utilisés dans l'algorithme de PSO sont identifiés empiriquement, à savoir, la taille de population et le critère d'arrêt. Le processus général de cette hybridation est donné par l'algorithme suivant (fig.3.2).

Initialiser les coefficients $c1$, $c2$ et C .

1. Initialiser les positions aléatoirement ainsi que la vitesse.
2. Evaluer la fitness pour chaque particule et calculer $\vec{p}g$.
3. Pour chaque particule x_i , la fitness courante $f(x_i)$ est comparée à celle précédente. Si $\Delta < 0$ alors x_i est la meilleure position.
4. La position et la vitesse de chaque particule sont mises à jour suivant l'équation

$$\begin{cases} \vec{v}_i(t) = \omega * \vec{v}_i(t-1) + c1 * \rho1 * (\vec{p}_i - \vec{x}_i(t-1)) + c2 * \rho2 * (\vec{p}g - \vec{x}_i(t-1)) \\ \vec{x}_i(t) = \vec{x}_i(t-1) + \vec{v}_i(t) \end{cases}$$
5. Calculer le nouveau $\vec{p}_i$ et $\vec{p}g$
6. Si les conditions d'arrêt ne sont pas satisfaites alors aller à l'étape 3 sinon arrêter l'algorithme.

Fig.3.2 Algorithme Otsu-PSO pour la segmentation.

6. Hybridation de l'algorithme PSO avec RS

Dans la littérature, le plus grand pourcentage de l'hybridation des métaheuristiques pour résoudre différents problèmes d'optimisation difficile est celui de coopération d'une recherche locale avec une recherche globale exploitant ainsi la caractéristique d'exploitation et celle d'exploration. Cette hybridation est qualifiée de HTH (High level Teamwork Hybrid) ou HRH (High level Relay Hybrid) selon le niveau et le mode de l'hybridation comme cela est défini dans le précédent chapitre. Dans notre travail, c'est l'hybridation HRH que nous avons adoptée pour optimiser la segmentation des images en niveaux de gris que nous avons utilisées. Il s'agit d'un algorithme hybride entre les PSO et le RS (fig.3.3). En effet, il a été montré que l'utilisation de l'algorithme des PSO est efficace car il permet de converger rapidement vers des optima locaux [12]. Cependant, dans cet algorithme, lorsqu'une particule trouve un optimum local, toutes les autres particules se regroupent autour de cet optimum et ne parviennent plus à s'en échapper. C'est pour cette raison que l'introduction du recuit simulé peut constituer une solution pour sortir des optima locaux en s'éloignant de la meilleure solution présente. Dans notre application, les particules correspondent aux seuils de l'image à segmenter et l'objectif est de trouver les seuils optimaux qui permettent de donner le seuillage optimal des images traitées tout en notant que la fonction objectif de l'algorithme est calculée d'après la méthode d'Otsu. L'algorithme hybride est composé des étapes suivantes (fig3.4).

1. Initialiser les coefficients $c1$, $c2$, C et T .
2. Initialiser les positions aléatoirement ainsi que la vitesse.
3. Evaluer la fitness pour chaque particule et calculer $\vec{p}g$.
4. Pour chaque particule x_i , la fitness courante $f(x_i)$ est comparée à celle précédente.
Si $\Delta < 0$ alors x_i est la meilleure position sinon la solution est acceptée selon la probabilité $P = \exp(-\Delta f / T)$.
5. La position et la vitesse de chaque particule sont mises à jour suivant l'équation

$$\begin{cases} \vec{v}_i(t) = \omega * \vec{v}_i(t-1) + c1 * \rho1 * (\vec{p}i - \vec{x}_i(t-1)) + c2 * \rho2 * (\vec{p}g - \vec{x}_i(t-1)) \\ \vec{x}_i(t) = \vec{x}_i(t-1) + \vec{v}_i(t) \end{cases}$$
6. La température T est ajustée suivant la formule $T = T * 0.9$.
7. Si les conditions d'arrêt ne sont pas satisfaites alors aller à l'étape 3 sinon arrêter l'algorithme.

Fig 3.4 Algorithme hybride de PSO-RS.

Nos algorithmes sont testés sur des images synthétiques Synth1, Synth2 et Square, ainsi que des images réelles Alumgrns, Fluocel et Bridge en niveaux de gris codées sur 8 bits de taille 256x256. Il est à noter que la segmentation de chaque image est effectuée suivant quatre et cinq classes.

7. Segmentation des images en fonction du nombre de classes

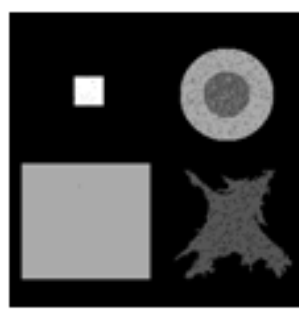
Afin de voir l'utilité de l'hybridation proposée ainsi que l'efficacité de la méthode d'Otsu dans le cas de multiseuillage, nous allons effectuer une segmentation des images réelles et synthétiques en quatre classes puis en cinq classes.

7.1 Segmentation des images en quatre classes

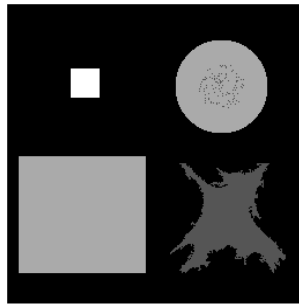
La segmentation des images réelles et synthétiques en quatre classes donne les résultats montrés aux figures suivantes.



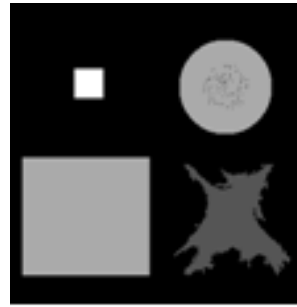
(a)



(b)

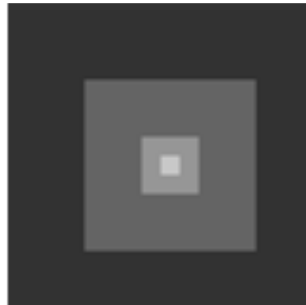


(c)

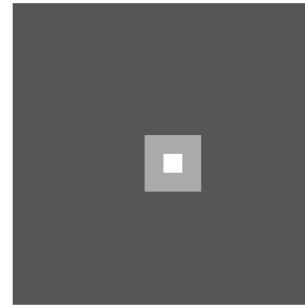


(d)

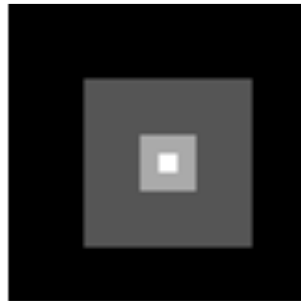
Fig.3.5 Segmentation de l'image Synth1. (a): image originale, (b): segmentation par la méthode RS-Otsu, (c): segmentation par la méthode PSO-Otsu, (d): segmentation par l'hybridation des méthodes.



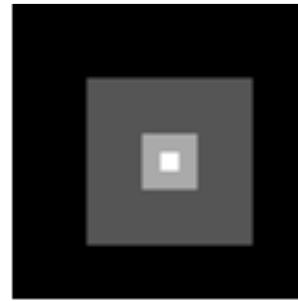
(a)



(b)

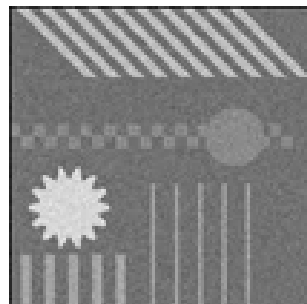


(c)

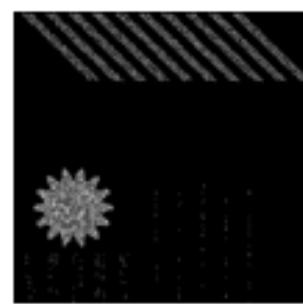


(d)

Fig.3.6 Segmentation de l'image Square. (a): image originale, (b): segmentation par la méthode RS-Otsu, (c): segmentation par la méthode PSO-Otsu, (d): segmentation par l'hybridation des méthodes.



(a)



(b)

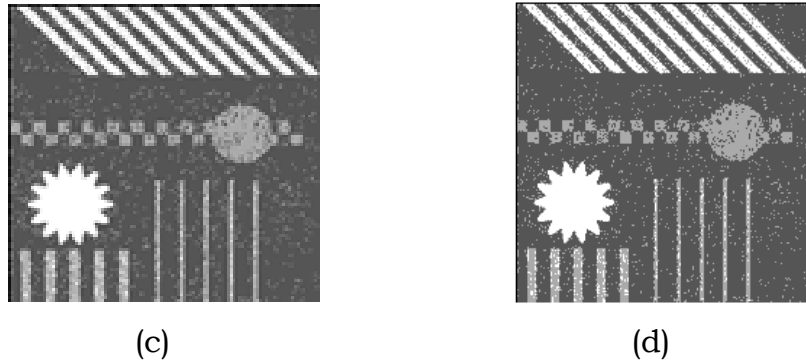


Fig.3.7 Segmentation de l'image Synth2. (a): image originale, (b): segmentation par la méthode RS-Otsu, (c): segmentation par la méthode PSO-Otsu, (d): segmentation par l'hybridation des méthodes.

Les figures qui suivent montrent la segmentation des trois images réelles en leur appliquant les trois méthodes différentes.

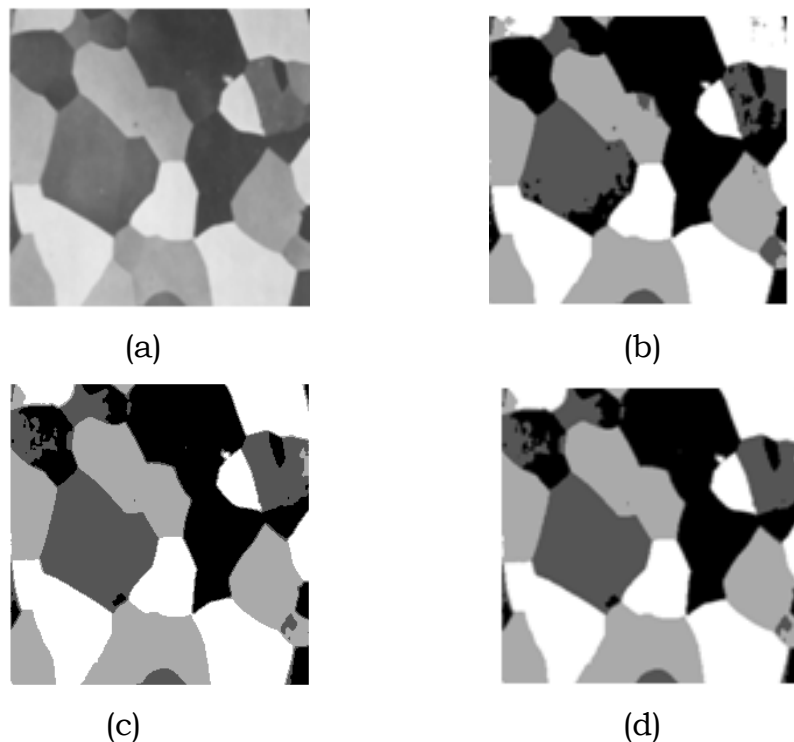


Fig.3.8 Segmentation de l'image Alumgrns. (a): image originale, (b): segmentation par la méthode RS-Otsu, (c): segmentation par la méthode PSO-Otsu, (d): segmentation par l'hybridation des méthodes.

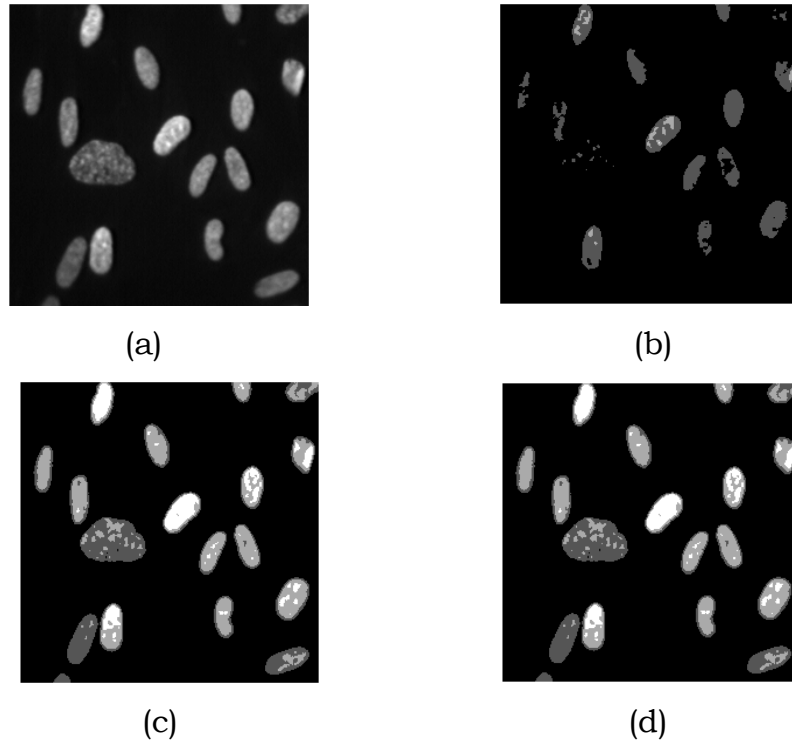


Fig.3.9 Segmentation de l'image Fluocel. (a): image originale, (b): segmentation par la méthode RS-Otsu, (c): segmentation par la méthode PSO-Otsu, (d): segmentation par l'hybridation des méthodes.

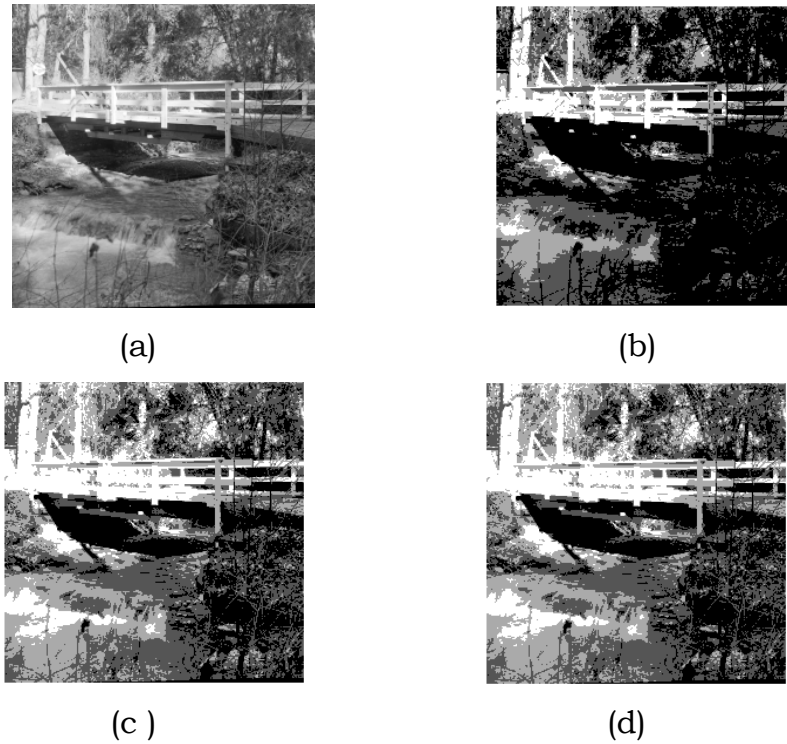


Fig.3.10 Segmentation de l'image bridge.(a): image originale, (b): segmentation par la méthode RS-Otsu, (c): segmentation par la méthode PSO-Otsu, (d): segmentation par l'hybridation des méthodes.

7.2 Segmentation des images en cinq classes

D'après les résultats obtenus, l'adaptation du recuit simulé permet d'avoir de bons résultats mais requiert un temps très important. Les figures suivantes montrent les résultats de segmentation des images synthétiques en appliquant les méthodes RS, PSO et PSO-RS intégrées dans la méthode d'Otsu.

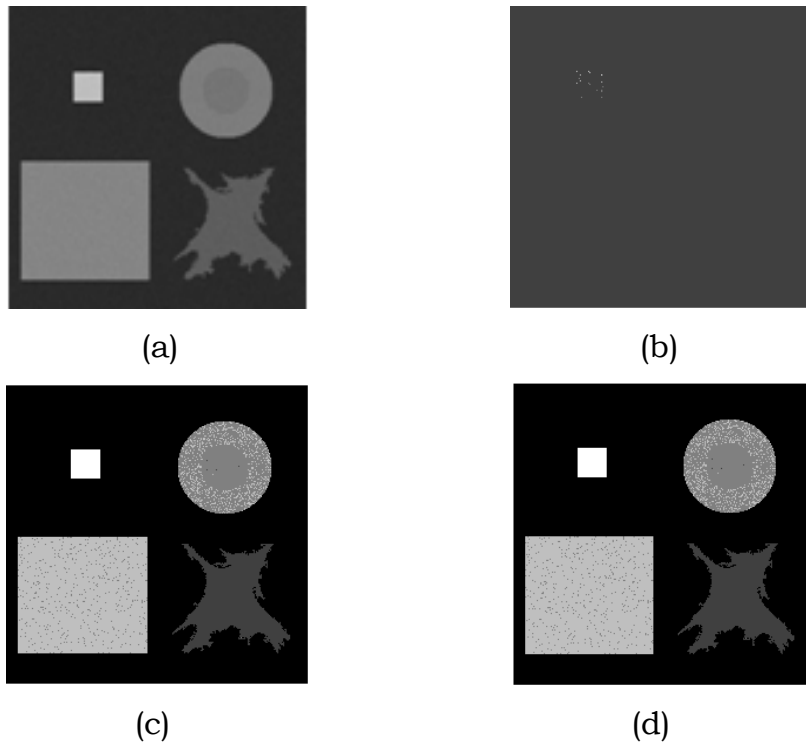
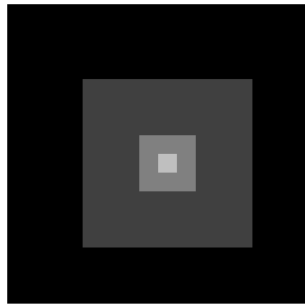
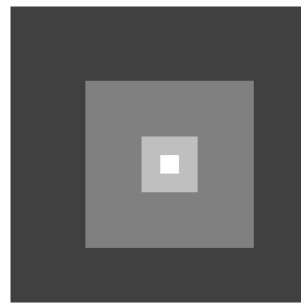


Fig 3.11 : Segmentation de l'image Synth1. (a) : image originale, (b) : segmentation par la méthode RS-Otsu, (c) : segmentation par la méthode PSO-Otsu, (d) : segmentation par l'hybridation des méthodes.



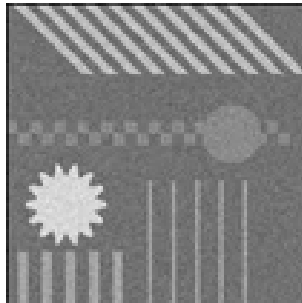


(c)



(d)

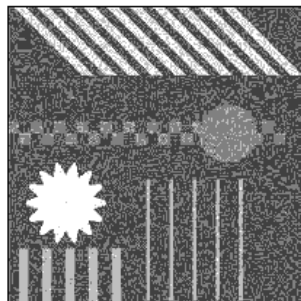
Fig 3.12 : Segmentation de l'image Square. (a) : image originale, (b) : segmentation par la méthode RS-Otsu, (c) : segmentation par la méthode PSO-Otsu, (d) : segmentation par l'hybridation des méthodes.



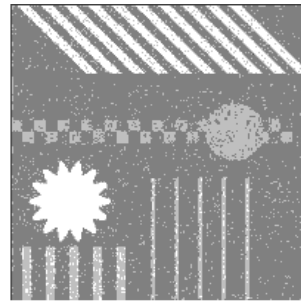
(a)



(b)



(c)



(d)

Fig 3.13 Segmentation de l'image Synth2. (a) : image originale, (b) : segmentation par la méthode RS-Otsu, (c) : segmentation par la méthode PSO-Otsu, (d) : segmentation par l'hybridation des méthodes.

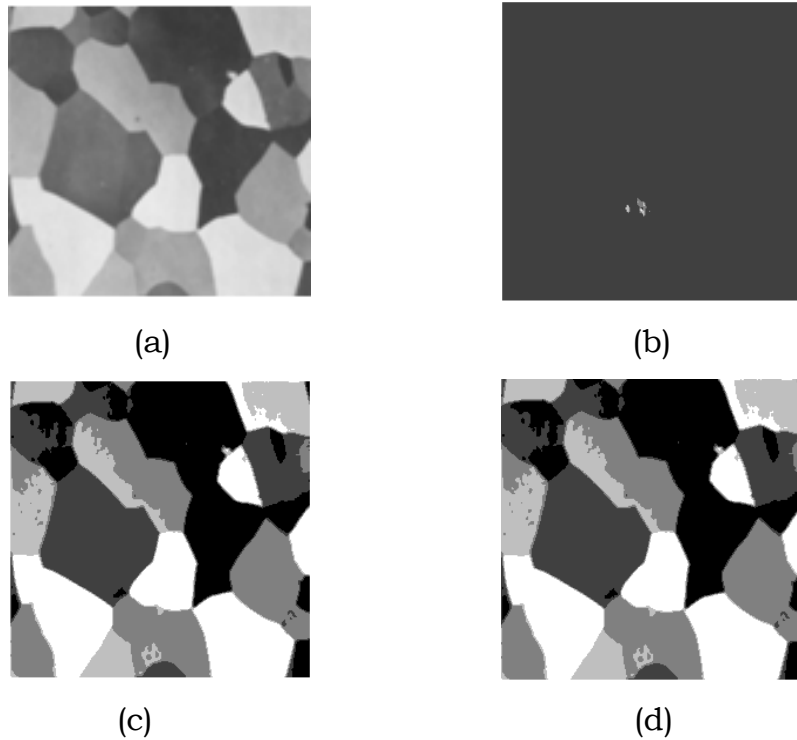


Fig.3.14 Segmentation de l'image Alumgrns. (a) : image originale, (b) : segmentation par la méthode RS-Otsu, (c) segmentation par la méthode PSO-Otsu, (d) : segmentation par l'hybridation des méthodes.

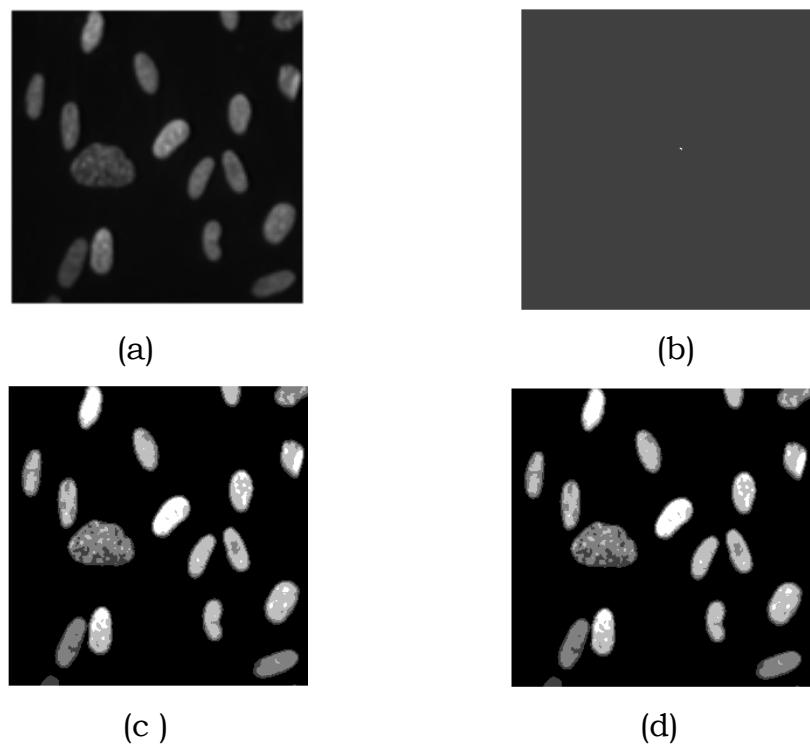


Fig.3.15 Segmentation de l'image Fluocel. (a) : image originale, (b) : segmentation par la méthode RS-Otsu, (c) : segmentation par la méthode PSO-Otsu, (d) : segmentation par l'hybridation des méthodes.

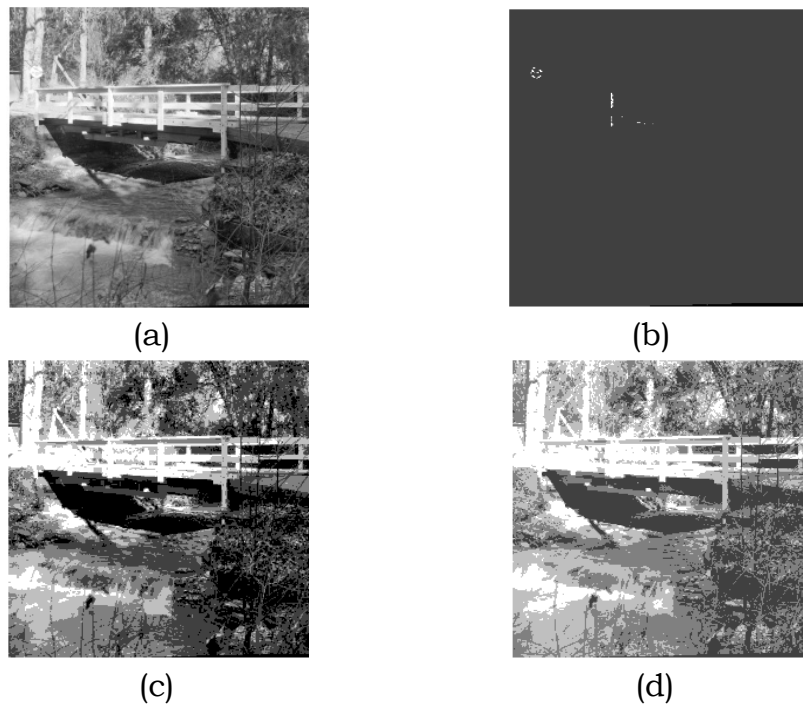


Fig.3.16 Segmentation de l'image bridge. (a) : image originale, (b) : segmentation par la méthode RS-Otsu, (c) segmentation par la méthode PSO-Otsu, (d) : segmentation par l'hybridation des méthodes.

8. Conclusion

Dans ce chapitre, nous avons effectué une segmentation en multiseuillage des images réelles et synthétiques dans le cas de quatre et cinq classes. Les résultats obtenus dans le cas de quatre classes sont satisfaisants ce qui conclue que l'intégration du recuit simulé à la méthode d'Otsu est applicable pour des classes de petites tailles. Cependant, quand la taille des classes augmente, cette méthode devient inefficace. Pour avoir de meilleurs résultats, nous avons intégré l'algorithme des essaims de particules à la méthode d'Otsu pour accélérer la recherche. Cette métaheuristique donne des résultats performants par rapport à la méthode précédente mais avec une optimisation locale de la fonction objectif. Pour pallier à ce problème, l'algorithme du recuit simulé et incorporé dans celui des essaims particulaire. Les résultats obtenus montrent que cette hybridation a

permi d'avoir une optimisation globale de la fonction objectif en un temps d'exécution très court en obtenant un meilleur seuillage des images traitées.

Conclusion générale

Le présent travail a visé à étudier les différentes techniques d'hybridation des métaheuristiques en se basant sur des travaux déjà effectués dans ce concept. Nous avons conclu que cette hybridation peut être réalisée de différentes manières. Plusieurs taxinomies des algorithmes d'hybridation ont été proposées. Celle de Talbi classe les algorithmes selon leurs différentes caractéristiques alors que celle de Puchinger et Raidi se concentre sur la classification des algorithmes hybridant une métaheuristique avec une méthode exacte. Dans la plupart des cas, les travaux proposant des métaheuristiques hybrides sont des programmes séquentiels. Cependant, les algorithmes hybrides parallèles deviennent très intéressants lorsque les problèmes traités sont de grandes tailles. Ainsi, il existe quand même un certain nombre d'études portant sur des algorithmes hybrides parallèles. Mais les algorithmes de coopérations méta/exacte parallèles sont peu nombreux. En effet, les grandes différences entre les deux types d'algorithmes rendent difficile la mise en œuvre d'une architecture parallèle.

Dans ce mémoire, nous avons proposé une méthode coopérative de mode relais de haut niveau (HRH) composée d'un algorithme des essaims de particules et d'une recherche locale qui est le recuit simulé afin d'équilibrer les deux stratégies d'intensification et de

diversification. La première approche (PSO) calcule, à chaque itération de l'algorithme principal, la fonction objectif minimale du problème proposé. Au sein de la deuxième approche (RS), cette fonction objectif est acceptée même si elle n'est pas optimale si une condition liée à la convergence de l'algorithme est vérifiée. Au cours de notre travail, nous avons montré l'intérêt de ce type d'hybridation, en proposant des critères de comparaison à des résultats en utilisant des méthodes non hybridées.

Enfin, les méthodes coopératives peuvent résoudre des problèmes d'optimisation dans différents domaines en améliorant significativement les résultats obtenus. La combinaison de métaheuristiques fournit généralement des algorithmes de recherche de haute performance pour de nombreuses classes de problèmes académiques et industriels. Le choix d'une technique d'hybridation diffère pratiquement d'une situation à l'autre et c'est en fonction de performances, et en fonction de contraintes économiques dont dispose l'utilisateur pour modéliser, programmer et tester son système, que se fera le choix de telle ou telle solution.

Des améliorations peuvent être apportées à notre travail. En effet, l'utilisation des fonctions multiobjectif *a priori* permettrait un choix judicieux des mécanismes de coopération afin de sélectionner dynamiquement la méthode la plus adaptée selon tel ou tel critère de convergence.

Références bibliographiques

- [1] Christelle Guéret, Christian Prins et Marc Sevaux : Programmation Linéaire, Eyrolles, 2000.
- [2] A.Sbihi : Les méthodes hybrides en optimisation combinatoire : Algorithmes exactes et heuristiques. Thèse de doctorat de l'université Paris-I Panthéon-Sorbonne. Année 2003.
- [3] Chandru V. and Rao M.R. Linear Programming, Chapter 31 in Algorithms and Theory of Computation Handbook, edited by M.J.Atallah, CRC Press 1999.
- [4] J.Dréo, A.Pétrowski, P.Siarry, E.Taillard : Métaheuristiques pour l'optimisation difficile. Edition Eyrolles, 2003.
- [5] C.H.Papadimitriou. The complexity of combinatorial optimization problems. PhD thesis, Princeton, 1976
- [6] S. Kirkpatrick, D.C.Gelatt, and M.P.Vechhil: Optimization by simulated annealing. Science, 220:671–680, May1983.
- [7] Glover, F: Tabu search—Part II, ORSA J. Comput. 2:4-32. 1990.
- [8] T.Feo et M.Resende: Greedy Randomised Adaptive Search procedure. Journal of Global Optimization, tome 42, pages 860-878, 1995.
- [9] K. Price, R. Storn, Differential evolution – a simple and efficient adaptive scheme for global optimization over continuous spaces, Technical Report, International Computer Science Institute, Berkley, 1995.
- [10] A.Colorni, M.Dorigo, V.Maniezzo : Distributed Optimization by Ant Colonies. Proceedings of ECAL'91-First European Conference on Artificial Life, édité par F.Varela, et al., pages 134_142.Elsevier Publishing, Paris, France, 1992.
- [11] Johann Dréo, Patrick Siarry : Métaheuristiques pour l'optimisation et auto-organisation dans les systèmes biologiques. Université de Paris XII Val-de-Marne.
- [12] KENNEDYJ., EBERHART R. C., « Particle swarm optimization »,Proc. IEEE Int.Conf. On Neural Networks, vol. IV, Piscataway, NJ: IEEE Service Center, 1995, p. 1942-1948.
- [13] Yann COOREN, Maurice CLERC et Patrick SIARRY: Optimisation par essaim particulière amélioré par hybridation avec un algorithme à estimation de distribution. Laboratoire Images, Signaux et Systèmes Intelligents, LiSSi, E.A. 3956 Université de Paris XII.
- [14] R.C. Eberhart, Y. Shi, Comparing inertia weights and constriction factors in particle swarm optimization, Proceedings of the IEEE Congress on Evolutionary Computation 1 (2000) 84–88.
- [15] M. Clerc, The swarm and the queen: towards a deterministic and adaptive particle swarm optimization, Proceedings of the IEEE Congress on Evolutionary Computation 3 (1999) 1951–1957.
- [16] M. Clerc, J. Kennedy, The particle swarm-explosion, stability, and convergence in a multimodal complex space, IEEE Transactions on Evolutionary Computation 6 (2002) 58–73.

- [17] E.Talbi : A taxonomy of hybrid metaheuristics . Journal of Heuristics, tome 8, pages 541-564, 2002.
- [18] F.Glover: Heuristics for integer programming using surrogate constraints. Decision Sciences, tome 8, n°1, pages 156-166, 1977.
- [19] J. J. Grefenstette : Incorporating problem specific knowledge into genetic algorithms, in *Genetic Algorithms and Simulated Annealing*, Davis, L. D. (Ed.), London: Pitman. (1987).
- [20] H.Mühlenbein, M.Gorges-Schleuter et O.Kramer : Evolution Algorithms in Combinatorial Optimization. Parallel Computing, tome 7, pages 65-88, 1988.
- [21] Martin, O.c. et S.W. Otto, Combining Simulated Annealing with Local Search Heuristics. Annals of Operations Research, 1996.63: pp. 57-75.
- [22] Stützle, T. et H. Hoos. MAX-MIN Ant System and Local Search for Combinatorial Optimization Problems, in 2nd International Conference on Metaheuristics, 1997, Sophia-Antipolis, France, pp. *191-193.
- [23] Gambardella, É. Taillard et M. Dorigo, Ant Colonies for the Quadratic Assignment Problem. Journal of the Operational Research Society, 1999.50: pp. 167-176.
- [24] McKendall, A.R. et J. Shang, Hybrid And Systems for the Dynamic Facility Layout Problem. Computers and Operations Research, 2006.33: pp.790-803.
- [25] Azimi, Z.N., Hybrid Heuristics for Examination Time tabling Problem. Applied Mathematics and Computation, 2005. 163(2): pp. 705-733.
- [26] J. Suh and D.van Gucht. Incorporating heuristic information in to genetic search. In 2nd International Conference on Genetic Algorithms, pages 100-107, 1987.
- [27] C. Fleurent and A. Ferland. Genetic hybrids for the quadratic assignment problem.DI-MACS Series in discrete Mathematics and Theoretical Computer Science, 16:173-188, 1994.
- [28] E.-G. Talbi, T.Muntean, and I.Samarandache. Hybridation des algorithmes génétiques avec la recherche tabou. In Evolution Artificielle (EA'94), 1994.
- [29] Lin, FT, c.y. Kao et c.c. Hsu. Incorporating Genetic Algorithms into Simulated Annealing, in Fourth International Symposium On Artificial Intelligence, 1991, pp. 290-297.
- [30] R.Tanese. Parallel genetic Algorithms for a Hypercube, in Proceedings of the Second International Conference on Genetic Algorithms, 1987, MIT, Cambridge, MA, USA, pp.177-183.
- [31] M.Hammami et K. Ghédira :Cooperation between Simulated Annealing, Tabu Search, X-over operator and Kernighan and Lin Heuristic for the K-Graph Partitioning Problem, in MLC2005, The 6th Metaheuristics International Conference, 2005, Vienna, Austria, pp. 480-485.
- [32] I. de Falco, R. del Balio, and E. Tarantino. An analysis of parallel heuristics for task allocation in multicomputers . Computing, 3(59):259-275, 1995.
- [33] I. de Falco, R. del Balio, E. Tarantino, and R. Vaccaro. Improving search by incorpora-ting evolution principles in parallel tabu search. In International Conference on Machine Learning, pages 823-828, 1994.
- [34] J. Kosa and D. Andre: Parallel genetic programming on a network of transputers. Technical report, Standford University, 1995. Technical Report CS-TR-95-1542.

- [35] H.M. Voigt, J. Born, and I. Santibanez-Koref: Modelling and simulation of distributed evolutionary search processes for function optimization. *Parallel Problem Solving from Nature*, 496 :373-380, 1990. LNCS.
- [36] A. Liefvooghe: Métaheuristiques pour l'optimisation multiobjectif, Approches coopératives, prise en compte de l'incertitude et application en logistique. Thèse de doctorat de l'université Lille 1 - Sciences et Technologies. Année 2009.
- [37] É.D.Taillard: Parallel Iterative Search Methods for Vehicle Routing Problems. *Networks*, 1993. 23: pp. 661-673.
- [38] É.D.Taillard, et S. Voss, POPMUSIC: Partial Optimization Metaheuristic Under Special Intensification Conditions. Dans C.C. Ribeiro, P.Hansen (eds), *Essays and Surveys in Metaheuristics*, Kluwer academic publishers, 2001: pp. 613-629.
- [39] M.Krueger : Méthodes d'analyse d'algorithmes d'optimisation stochastiques à l'aide d'algorithmes génétiques. 1993, École Nationale Supérieure des Télécommunications: Paris, France.
- [40] F.Abbattista., N. Abbattista et L. Caponetti. An Evolutionary and Cooperative Agent Model for Optimization, in *IEEE International Conference on Evolutionary Computation ICEC'95*, 1995, Perth, Australia, pp. 668-671.
- [41] K.Shahookar et P. Mazumder: A Genetic Approach to Standard Cell Placement Using Meta-Genetic Parameter Optimization. *IEEE Transactions on Computer-Aided Design*, 1990. 9(5): pp. 500-511.
- [42] P. Augerat, J. M. Belenguer, E. Benavent, A. Corber'n, and D. Naddef: Separating capacity constraints in the CVRP using tabu search. *European Journal of Operational Research*, 106 (2-3) :546-557, April 1998.
- [43] C. Cotta and J. M. Troya: Embedding branch and bound within evolutionary algorithms. *Applied Intelligence*, 18:137-153, 2003.
- [44] C. A. R. Jahuira. Hybrid genetic algorithm with exact techniques applied to tsp. In *Second international workshop on Intelligent systems design and application*, pages 119-124. Dynamic Publishers, 2002.
- [45] K. Kostikas and C. Fragakis. Genetic programming applied to mixed integer programming. In *EuroGP 2004*, volume 3003 of *Lecture Notes in Computer Science*, pages 113-124, 2004.
- [46] R. Bent and P. Van Hentenryck. A two-stage hybrid local search for the vehicle routing problem with time windows. *Technical Report Tech Report CS-01-06*, Brown University, USA, September 2001.
- [47] M. C. Portmann, A. Vignier, D. Dardihac, and D. Dezalay: Branch and bound crossed with ga to solve hybrid flowshops. *European Journal of Operational Research*, 107(2):389-400, 1998.
- [48] A. Chabrier, E. Danna, and C. L. Pape. Coopération entre génération de colonnes sans cycle et recherche locale appliquée au routage de véhicules. In *JNPC*, 2002.
- [49] Puchinger, J. et G.R. Raidl: Combining Metaheuristics and Exact Algorithms in Combinatorial Optimization: A Survey and Classification, in *Proceedings of the First International Work-Conference on the Interplay Between Natural and Artificial Computation*, 2005, Springer (Ed), Las Palmas, Spain, LNCS, pp. 41-53.
- [50] D.Applegate, R. Bixby, V. Chvátal et W. Cook: On the Solution of Travelling Salesman Problems. *Documenta Mathematica*, Extra Volume ICM III, 1998: pp. 645-656.

- [51] G.W.Klau, I.Ljubic, A.Moser, P. Mutzel, P. Neuner, U. Pferschy, G. Raidl et R.Weiskircher. Combining a Memetic Algorithms with Integer Programming to Solve the Prize-Collecting Steiner Tree Problem, in Genetic and Evolutionary Computation - GECCO, 2004, pp. 1304-1315.
- [52] Feltl, H. et G.R. Raidl. An Improved Hybrid Genetic Algorithm for the Generalized Assignment Problem, in Proceedings of the 2004 ACM Symposium on Applied Computing, 2004, Nicosia, Cyprus, ACM Press, pp. 990-995.
- [53] M.Palpant, C. Artigues et P.Michelon, LSSPER: Solving the Resource-Constrained Project Scheduling Problem with Large Neighbourhood Search. Annals of Operations Research, 2004.131: pp. 237-257.
- [54] D.Applegate et W.Cook: A Computational Study of the Job-Shop Scheduling Problem. ORSA Journal on Computing, 1991. 3(2): pp. 149-156.
- [55] C. Pessan, J.-L. Bouquard, et E. Néron, 2007. Genetic Branch-and-Bound or Exact Genetic Algorithm ? Artificial Evolution, 136-147.
- [56] Talukdar, S., L. Baerentzen, A. Gove et P.d. Souza, Asynchronous Teams: Cooperation Schemes for Autonomous Agents. Journal of Heuristics, 1998. 4: pp. 295-321.
- [57] Souza, P.d., Asynchronous Organizations for Multi-Algorithm Problems, in Dept. of Electrical and Computer Engineering. 1993, Carnegie Mellon University: Pittsburgh, PA.
- [58] S.Y.Chen, S.N. Talukdar et N.M. Sadeh: Job Shop Scheduling by a Team of Asynchronous Agents, in Proceedings of the IJCAI '93 Workshop on Knowledge-Based Production, Scheduling and Control, 1993, Chambery, France.
- [59] C.K.Tsen, Solving Train Scheduling Problems Using A-Teams, Thèse de doctorat, in Dept. of Electrical and Computer Engineering. 1995, Carnegie Mellon University: Pittsburgh, PA.
- [60] Burke, EX., P.I. Cowling et R Keuthen. Effective Local And Guided Variable Neighborhood Search Methodsfor the Asymmetric Travelling Salesman Problem., in Applications of Evolutionary Computing: Evo Workshops 2001, volume 2037 of LNCS, 2001, S.a. Heidelberg (Ed), Springer, pp. 203-212.
- [61] Raidi, G.R. An Improved Genetic Algorithm for the Multiconstrained 0-1 Knapsack Problem, in Proceedings of the 1998 IEEE International Conference on Evolutionary Computation, 1998, D.B. Fogel (Ed), IEEE Press, pp. 207-211.
- [62] P.C. Chu et J.E. Beasley, A Genetic Algorithm for the Multidimensional Knapsack Problem. Journal of Heuristics, 1998. 4(1): pp. 63-86.
- [63] RK.Congram, C.N. Potts et S.L.v.d. Velde: An Iterated Dynasearch Algorithm for the Single-Machine Total Weighted Tardiness Scheduling Problem. INFORMS Journal on Computing, 2002. 14(1): pp. 52-67.
- [64] C.Cotta, et J.M. Troya, Embedding Branch and Bound Within Evolutionary Algorithms. Applied Intelligence, 2003. 18: pp.137-153.
- [65] M. Khichane, P. Albert, et C. Solnon, 2008.Programmation par contraintes avec des fournis. Journées Francophones de Programmation par Contraintes.
- [66] D.L. Woodruff; A Chunking Based Selection Strategy for Integrating Meta-heuristics with Branch and Bound, in Metaheuristics: Advances and Trends in Local Search Paradigms for Optimization, 1999,Kluwer Academic Publishers, pp. 499-511.

- [67] French, AP., AC. Robinson et J.M. Wilson, Using a Hybrid Genetic-Algorithm/Branch and Bound Approach to Solve Feasibility and Optimization Integer Programming Problems. *Journal of Heuristics*, 2001. 7: pp. 551-564.
- [68] I. Dumitrescu and T. Stützle. Combinations of local search and exact algorithms. In *EvoWorkshops 2003*, pages 211-223, 2003.
- [69] C.Cotta, E.-G. Talbi et E. Alba: Parallel Hybrid Metaheuristics, in *Parallel Metaheuristics: A New Class of Algorithms*, E. Alba (Ed.) 2005, Wiley-Interscience. pp 347-370.
- [70] P. Angeline. (1998): Evolutionary optimization versus particle swarm optimization, *Proceedings of the Evolutionary Programming VII*, pp. 601-610, 1998, Springer.
- [71] V. Miranda, N. Fonseca. (2002): New evolutionary particle swarm algorithm applied to voltage/VAR control, *Proceedings of the 14th Power Systems Computation Conference*, 2002, pp. 1-6, Sevilla, Spain.
- [72] J. Robinson, S. Sinton, Y.R. Samii: Particle swarm, genetic algorithm, and their hybrids: optimization of a profiled corrugated horn antenna, in: *Proceedings of the IEEE International Symposium in Antennas and Propagation Society*, 2002, pp. 314–317.
- [73] T. Krink, M. Løvbjerg, The lifecycle model: combining particle swarm optimization, genetic algorithms and hill climbers, *Proceedings of the Parallel Problem Solving From Nature (2002)* 621–630.
- [74] E.A. Grimaldi, F. Grimaccia, M. Mussetta, P. Pirinoli, R.E. Zich, A new hybrid genetical – swarm algorithm for electromagnetic optimization, in: *Proceedings of International Conference on Computational Electromagnetics and its Applications*, Beijing, China, 2004, pp. 157–160.
- [75] Gandelli, F. Grimaccia, M. Mussetta, P. Pirinoli, R.E. Zich, Genetical swarm optimization: a new hybrid evolutionary algorithm for electromagnetic application, in: *Proceedings of the 18th International Conference on Applied Electromagnetics, ICECcom 2005, Dubrovnik, Croatia, 2005*, pp. 269–272.
- [76] Gandelli, F. Grimaccia, M. Mussetta, P. Pirinoli, R.E. Zich, Genetical swarm optimization: an evolutionary algorithm for antenna design, *Journal of AUTOMATIKA* 47 (3-4) (2006) 105–112.
- [77] M. Settles, T. Soule: Breeding swarms: a GA/PSO hybrid, *Proceedings of Genetic and Evolutionary Computation Conference (2005)* 161–168.
- [78] Y.T. Kao, E. Zahara: A hybrid genetic algorithm and particle swarm optimization for multimodal functions, *Applied Soft Computing* 8 (2008) 849–857
- [79] C-F. Juang, A hybrid of genetic algorithm and particle swarm optimization for recurrent network design, *IEEE Transactions On Systems, Man, And Cybernetics-Part B: Cybernetics* 34 (2004) 997–1006.
- [80] M. Jian, Y. Chen, Introducing recombination with dynamic linkage discovery to particle swarm optimization, *Proceedings of the Genetic and Evolutionary Computation Conference (2006)* 85–86.
- [81] H. Kim: Improvement of genetic algorithm using PSO and Euclidean data distance, *International Journal of Information Technology* 12 (2006) 142–148.

- [82] A. Mohammadi, M. Jazaeri: A hybrid particle swarm optimization-genetic algorithm for optimal location of SVC devices in power system planning, in: Proceedings of 42nd International Universities Power Engineering Conference, 2007, pp.1175–1181.
- [83] N. Ru, Y. Jianhua, A GA and particle swarm optimization based hybrid algorithm, in: Proceedings of the IEEE Congress on Evolutionary Computation, 2008, pp. 1047–1050.
- [84] A. Shunmugalatha, S.M.R. Slochanal: Optimum cost of generation for maximum loadability limit of power system using hybrid particle swarm optimization, International Journal of Electrical Power & Energy Systems 30 (2008) 486–490.
- [85] S. Jeong, S.Hasegawa, K. Shimoyama, S. Obayashi: Development and investigation of efficient GA/PSO-hybrid algorithm applicable to real-world design optimization, IEEE Computational Intelligence Magazine (2009)36–44.
- [86] F. Valdez, P. Melin, O. Castillo, Evolutionary method combining particle swarm optimization and genetic algorithms using fuzzy logic for decision making, in: Proceedings of the IEEE International Conference on Fuzzy Systems, 2009, pp. 2114–2119.
- [87] R.F. AbdelKader: Genetically improved PSO algorithm for efficient data clustering, in: Proceedings of International Conference on Machine Learning and Computing, 2010, pp. 71–75.
- [88] T. Hendtlass: A Combined Swarm differential evolution algorithm for optimization problems, in: Proceedings of 14th International Conference on Industrial and Engineering Applications of Artificial Intelligence and Expert Systems, Lecture Notes in Computer Science, vol. 2070, Springer Verlag, 2001, pp. 11–18.
- [89] S. Kannan, S.M.R. Slochanal, P. Subbaraj, N.P. Padhy, Applications of particle swarm optimization techniques and its variants to generation expansion planning, Electric Power Systems Research 70 (2004) 203–210.
- [90] H. Talbi, M. Batouche: Hybrid particle swarm with differential evolution for multimodal image registration, Proceedings of the IEEE International Conference on Industrial Technology 3 (2004) 1567–1573.
- [91] T. Hendtlass, M. Randall, A survey of ant colony and particle swarm meta-heuristics and their application to discrete optimization problems, in: Proceedings of the Inaugural Workshop on Artificial Life, 2001, pp. 15–25.
- [92] W.J. Zhang, X.F. Xie, DEPSO: hybrid particle swarm with differential evolution operator, in: Proceedings of the IEEE International Conference on Systems, Man and Cybernetics (SMCC), Washington DC, USA, 2003, pp. 3816–3821.
- [93] Z.-F.Hao, G.-H.Gua, H.Huang: A particle swarm optimization algorithm with differential evolution, in: Proceedings of Sixth International Conference on Machine Learning and Cybernetics, 2007, pp. 1031–1035.
- [94] G-N.Jose, E.Alba, J.Apolloni: Particle swarm hybridized with differential evolution: black box optimization benchmarking for noisy functions, in: Proceedings of International Conference Genetic and Evolutionary Computation, 2009, pp. 2343–2350.
- [95] W.Xu, X.Gu: A hybrid particle swarm optimization approach with prior crossover differential evolution, in: Proceedings of ACM/SIGEVO Summit on Genetic and Evolutionary Computation, 2009, pp. 671–678.

- [96] M. Pant, R. Thangaraj, A. Abraham, DE-PSO: a new hybrid meta-heuristic for solving global optimization problems, *New Mathematics and Natural Computation*, Accepted for publication, 2009
- [97] S. Khamsawang, P. Wannakarn, S. Jiriwibhakorn Hybrid PSO-DE for solving the economic dispatch problem with generator constraints, in: *Proceedings of the IEEE International Conference on Computer and Automation Engineering*, vol. 5, 2010, pp. 135–139.
- [98] P.S. Shelokar, P. Siarry, V.K. Jayaraman, B.D. Kulkarni, Particle swarm and ant colony algorithms hybridized for improved continuous optimization, *Applied Mathematics and Computation* 188 (2007) 129–142.
- [99] D.Y. Sha, C.-Y. Hsu: A hybrid particle swarm optimization for job shop scheduling problem, *Computers & Industrial Engineering* 51 (2006) 791–808.
- [100] Q. He, L. Wang: A hybrid particle swarm optimization with a feasibility-based rule for constrained optimization, *Applied Mathematics and Computation* 186 (2007) 1407–1422.
- [101] Wei, Z. He, W. Pei, Swarm directions embedded in fast evolutionary programming, in: *Proceedings of the World Congress on Computational Intelligence*, vol. 2, 2002, pp. 1278–1283.
- [102] Q.J. Guo, H.B. Yu, A.D. Xu, A hybrid PSO-GD based intelligent method for machine diagnosis, *Digital Signal Processing* 16 (2006) 402–418.
- [103] W. Mo, S.-U. Guan, S.K. Puthusserypady, A novel hybrid algorithm for function optimization: particle swarm assisted incremental evolution strategy, *Studies in Computational Intelligence* 75 (2007) 101–125.
- [104] B. Firouzi, T. Niknam, M. Nayeripour, A new evolutionary algorithm for cluster analysis, *World Academy of Science, Engineering, and Technology* 36 (2008) 605–609.
- [105] X.Li, H.Xu, Z.Cheng: One immune simplex particle swarm optimization and its application, in: *Proceedings of the Fourth International Conference on Natural Computation*, 2008, pp. 331–335.
- [106] H.-W.Ge, L.Sun, Y.-C. Liang, F.Qian: An effective PSO and AIS-based hybrid intelligent algorithm for job-shop scheduling, *IEEE Transactions on Systems, Man, And Cybernetics—Part A: Systems and Humans* 38 (2008) 358–368.
- [107] Kavehand, S. Talatahari, Particle swarm optimizer, ant colony strategy and harmony search scheme hybridized for optimization of truss structures, *Computer Structure* 87 (2009) 267–283.
- [108] R.Thangaraj, M.Pant, A.Abraham, P.Bouvry: Particle swarm optimization: Hybridization perspectives and experimental illustrations. *Applied Mathematics and Computation*. 2010.
- [109] A. Nakib : Conception de métaheuristiques d’optimisation pour la segmentation d’images. Application à des images biomédicales. Thèse de doctorat, 2007.
- [110] ADEL ZIADI : Analyse d’images visibles et proche infrarouges : Contribution à l’évaluation non-destructives du persillage dans la viande du bœuf. Thèse de doctorat département de Génie Electrique et de Génie Informatique. FACULTÉ DES SCIENCES ET DE GÉNIE UNIVERSITÉ LAVAL.QUÉBEC. Année 2010.

- [111] S.Ouadfel, M.Batouche : Des fourmis pour la segmentation des images. Département informatique, Université de Batna, Groupe de vision laboratoire LIRE Université de Constantine.
- [112] L.JOURDAN : Métaheuristiques Coopératives :du déterministe au stochastique. Habilitation à Diriger les Recherches de l'Université Lille I Année 2010.
- [113] J. B. MacQueen. Some methods for classification and analysis of multivariate observations. In Proceedings of 5th Berkeley Symposium on Mathematical Statistics and Probability, pages 281-297, 1967.
- [114] T. Minka. "Expectation-maximization as lower bound maximization.", 1998.
- [115] Fabrice Muhlenbach : Méthode de clustering par graphe de voisinage. Université de Lyon, Université Jean Monnet UMR CNRS 5516, Laboratoire Hubert Curien SAINT-ETIENNE, France. Laboratoire ERIC, Université Lyon 2, le 27 novembre 2008.
- [116] Lengrand-Lambert, J. Fusion de la segmentation et de la classification automatique des images sonar. Technical report, Projet industriel, ENSIETA (2009).
- [117] Horaud, R. P. and Monga, O : Vision par ordinateur: outils fondamentaux. Paris: Hermes, 1995.
- [118] S. Olafsson, X. Li, and S. Wu. Operations research and data mining. European Journal of Operational Research, 187(3) :1429-1448, 2008.
- [119] L. Jourdan, C. Dhaenens, and E.-G. Talbi. Using datamining techniques to help meta-heuristics : A short survey. In Hybrid Metaheuristics (HM'06), volume LNCS 4030, pages 57-69. Springer Berlin / Heidelberg, 2006. Acceptance rate : 31%.
- [120] Feng, D., et al., Infrared image segmentation with 2-D maximum entropy method based on particle swarm optimization (PSO). Pattern Recognition Letters. 2005, Vol. 26, 5, pp. 597 -603.
- [121] Gonzalez, R.C., Woods, R.E., Digital image processing using Matlab. Pearson Prentice Hall, 2004.
- [122] Sezgin, M. and Sankur, B., Survey over image thresholding techniques and quantitative performance evaluation. J. of Electronic Imaging. 2004, Vol.13,1, pp. 146-165.
- [123] N. Otsu, A threshold selection method for grey level histograms. IEEE Transactions on System, Man and Cybernetics SMC vol. 9, n°1, pp. 62-66, 1979.
- [124] J. N. Kapur, P. K. Sahoo, and A. C. K. Wong, A new method for gray-level picture thresholding using the entropy of the histogram. Computer Vision, Graphics and Image Processing, vol. 29, pp. 273-285. 1985.
- [125] Y. Lahfi: Segmentation d'images radiologiques en temps réel dans le cadre d'un système d'aide à la pose d'une prothèse endo vasculaire. Thèse de doctorat, Université Paris 12, Octobre 2006, Créteil.
- [126] Y.Qiao, et al: Thresholding based on variance and intensity contrast. Pattern Recognition Letters. 2007, Vol. 40, pp. 596 -608.